A Survey of Distributed Consensus Protocols for Blockchain Networks

Yang Xiao, *Student Member, IEEE*, Ning Zhang, *Member, IEEE*, Wenjing Lou, *Fellow, IEEE*, and Y. Thomas Hou, *Fellow, IEEE*

***Abstract*—Since the inception of Bitcoin, cryptocurrencies and the underlying blockchain technology have attracted an increasing interest from both academia and industry. Among various core components, consensus protocol is the defining technology behind the security and performance of blockchain. From incremental modifications of Nakamoto consensus protocol to innovative alternative consensus mechanisms, many consensus protocols have been proposed to improve the performance of the blockchain network itself or to accommodate other specific application needs. In this survey, we present a comprehensive review and analysis on the state-of-the-art blockchain consensus protocols. To facilitate the discussion of our analysis, we first introduce the key definitions and relevant results in the classic theory of fault tolerance which help to lay the foundation for further discussion. We identify five core components of a blockchain consensus protocol, namely, block proposal, block validation, information propagation, block finalization, and incentive mechanism. A wide spectrum of blockchain consensus protocols are then carefully reviewed accompanied by algorithmic abstractions and vulnerability analyses. The surveyed consensus protocols are analyzed using the five-component framework and compared with respect to different performance metrics. These analyses and comparisons provide us new insights in the fundamental differences of various proposals in terms of their suitable application scenarios, key assumptions, expected fault tolerance, scalability, drawbacks and trade-offs. We believe this survey will provide blockchain developers and researchers a comprehensive view on the state-of-the-art consensus protocols and facilitate the process of designing future protocols.**

***Index Terms*—Blockchain, distributed consensus, fault tolerance, protocol design.**

I. Introduction

SINCE Bitcoin's inception in late 2008, cryptocurrencies and the underlying blockchain technology have piqued great interest from the financial industry and society as a whole. Blockchain is widely cited as a fully decentralized system and a secure-by-design technology. The blockchain itself is a database that keeps track of all transactions occurred in the network and is replicated at every participating node. It essentially realizes a distributed ledger without relying on a central authority to bootstrap the trust among participants or to clear the transactions. It also does not assume trust among the participating nodes. Blockchain is meant to enable trusted computation among a group of mutually distrustful participants. On the other hand, blockchain is also known for providing trustworthy immutable record keeping service. The block data structure adopted in a blockchain embeds the hash of the previous block in the next block generated. The use of hash chain ensures that data written on one block can not be modified without invalidating later blocks. In addition, a public blockchain system supports third-party auditing and some blockchain systems support a high level of anonymity, that is, a user can transact online using a pseudonym without revealing his/her true identity.

The security properties promised by blockchain is unprecedented and truly inspiring. Pioneering blockchain systems such as Bitcoin have greatly impacted the digital payment world. It is envisioned that blockchain technology and applications built on top of it will revolutionize a broad array of financial service industries as well as non-financial sectors. Among the many technical components that a blockchain system is composed of, the distributed consensus protocol is the key technology that enables blockchain's decentralization, or more specifically, that ensures all participants agree on a unified transaction ledger without the help of a central authority. The distributed consensus protocol specifies message passing and local decision making at each node. Various design choices in the consensus protocol can greatly impact a blockchain system's performance, including its transaction capacity, scalablity, and fault tolerance.

The Nakamoto consensus protocol [1] is the protocol implemented in the Bitcoin network. With the help of this consensus protocol, Bitcoin became the first digital currency system to resist double-spending attacks in a decentralized peer-to-peer network of little trust. As the Bitcoin network continues to grow, Nakamoto consensus has encountered several performance bottlenecks and sustainability problems. Researchers in blockchain communities have raised the following concerns on Nakamoto consensus and particularly its proof-of-work (PoW) mining mechanism: 1) unsustainable energy consumption, 2) low transaction capacity and poor scalability, 3) long-term security concerns as mining rewards diminish. For instance, the Bitcoin

Manuscript received April 7, 2019; revised October 14, 2019 and November 29, 2019; accepted January 13, 2020. Date of publication January 28, 2020; date of current version May 28, 2020. This work was supported in part by U.S. National Science Foundation under Grant CNS-1916902 and Grant CNS-1916926. *(Corresponding author: Wenjing Lou.)*

Yang Xiao and Y. Thomas Hou are with the Bradley Department of Electrical and Computer Engineering, Virginia Polytechnic Institute and State University, Blacksburg, VA 24061 USA (e-mail: xiaoy@vt.edu; thou@vt.edu).

Ning Zhang is with the Department of Computer Science and Engineering, Washington University in St. Louis, St. Louis, MO 63130 USA (e-mail: zhang.ning@wustl.edu).

Wenjing Lou is with the Department of Computer Science, Virginia Polytechnic Institute and State University, Blacksburg, VA 24061 USA (e-mail: wjlou@vt.edu).

Digital Object Identifier 10.1109/COMST.2020.2969706

1553-877X © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

network currently consists of roughly ten thousand nodes [2], while the maximum transaction capacity of Bitcoin is 7 transactions per second (TPS) and can be increased to at most 25 TPS by tuning protocol parameters without jeopardizing consensus safety [3]. In contrast, the VISA network consists of 50 million participants and can handle up to 65,000 TPS [4]. A single Bitcoin transaction (November 2019) consumes the equivalent amount of electricity that would power 21 average U.S. households for one day [5].

In response to the above performance limitations of PoW mining, blockchain researchers have been investigating new block proposing mechanisms such as proof of stake (PoS), proof of authority (PoA), and proof of elapsed time (PoET) which do not require computation-intensive mining, thus effectively reducing energy consumption. In some cases, cryptographic methods can be used to establish trust among nodes, enabling the use of more coordinated block proposing schemes such as round-robin and committee-based block generation. Appropriate incentives that will continue to encourage honest participation in the blockchain network is another key component of consensus protocol. Therefore, alternative block proposing schemes are often accompanied by a new incentive mechanism that promotes participation fairness and increases overall system sustainability. Popular blockchain consensus protocols encompassing these ideas include Peercoin [6], Bitcoin-NG [7], Ourosboros (Cardano) [8], Snow White [9], and EOSIO [10], POA Network [11], etc.

Besides block proposing and incentive mechanisms, researchers have been seeking solutions from the prior wisdom—primarily classical Byzantine fault tolerant (BFT) consensus and secure multi-party computation (MPC)—for efficient block finalization methods. For example, the state machine replication (SMR) based BFT consensus algorithms have great potential in permissioned blockchain networks operated with static and revealed identities, of which TenderMint [12], Algorand [13], Casper FFG [14], and Hyperledger Fabric [15] are well-known use cases. Moreover, asynchronous consensus protocols such as HoneyBadgerBFT [16] and BEAT [17] were proposed to provide robust block finalization under severe network conditions with uncertain message delays.

Our Contribution: With more blockchain consensus mechanisms being proposed, there is a pressing need to analyze and compare them in a formal and cohesive manner. In this survey we present a comprehensive review and analysis of the state-of-the-art blockchain consensus protocols and their development history, with a special focus on their performance, fault tolerance, and security implications. Our information sources include academic papers, consensus protocol white papers, official documentation and statistics websites of cryptocurrencies. Specifically, our survey features the following contributions:

1) providing a background of classical distributed consensus research, including partially synchronous and asynchronous BFT protocols that are applicable to blockchain consensus;
2) reviewing a broad array of blockchain consensus protocols with a proposed five-component framework and analyzing their design philosophy and security issues;
3) identifying four classes of proof of stake (PoS) based consensus protocols and providing algorithmic abstractions for them;
4) comparing all mentioned consensus protocols with respect to the five-component framework, fault tolerance, and transaction processing capability.
5) providing a succinct tutorial on blockchain consensus protocol design with respect to the security-decentralization-scalability trilemma.

The remaining part of this survey is organized as follows. Section II reviews related surveys and tutorials on blockchain consensus protocols. Section III provides a background of classical fault tolerant consensus in distributed systems. Several legacy BFT consensus protocols designed for both partially synchronous and asynchronous networks are introduced. Section IV presents the basic framework of blockchain and the consensus goals, and introduces the five essential components of a blockchain consensus protocol. Section V focuses on the well-known Nakamoto consensus protocol, the defining technology of Bitcoin, and its vulnerabilities and improvement ideas. Section VI provides a systematic view of the PoS protocols, the most promising competitors to the PoW-based Nakamoto consensus for public blockchain. Section VII discusses alternative consensus protocols usable under specific application scenarios. Section VIII compares all blockchain consensus protocols studied and summarizes their design philosophy. Section IX discusses the paradigm shift in consensus protocol design and provides a succinct tutorial. Section X concludes the paper.

II. Previous Surveys and Tutorials

The comparison study by Vukolić [18] treats two genres of blockchain consensus protocols, namely PoW-based and BFT-based, with respect to transaction throughput, scalability limits, consensus finality, and security implications. PoW-based protocols scale well with network size and are suited for permissionless blockchains, but yield very limited throughput and long confirmation latency. This is due to the security implication of their lack of consensus finality and limited capacity of raising block frequency and block size. In comparison, BFT protocols have built-in consensus finality and achieve much higher transaction capacity, but incur high messaging complexity per block ($O(N^2)$ versus PoW's $O(N)$, N is network size) and need a permissioned network for identity management. As a result they do not scale well with network size. Besides remarking their differences, this paper also explores how the hybrid use of BFT and PoW can enhance the performance of established blockchain systems and provides tentative solutions to improve blockchain scalability.

Cachin and Vukolić [19] gives an overview of thirteen prominent consensus protocols designed for permissioned blockchain platforms along with their fault tolerance and security properties. The authors also make a powerful statement that the design of blockchain consensus protocols should follow the rigor established in prevailing wisdom of cryptography, security, and distributed systems, rather than in

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

an ad hoc manner. This is particularly true when the consensus protocol regulates significant financial values and societal trust. However, they do not provide a methodology to combine the prevailing wisdom in order to design a consensus protocol for specific needs.

The work by Bano *et al.* [20] is the first well-structured survey of blockchain consensus protocols. It identifies three classes of consensus protocols based on committee formation and block proposing rules: 1) PoW, 2) proof of X (PoX) alternatives to PoW, and 3) hybrid consensus protocols that take advantage of classical distributed consensus techniques. This paper emphasizes the role of committee in hybrid consensus protocols. General discussions on the formation and configuration of committee and possible solutions to multi-committee (i.e., sharding-based) consensus are provided. This paper also presents an evaluation framework that takes into account the protocol safety (censorship resistance, DoS resistance and fault tolerance) and performance (throughput and latency). Notably, this paper is the closest work to our survey in terms of classification of block proposing mechanisms. However, other protocol components need finer classification and the variety of PoS proposals deserves a more comprehensive treatment.

Wang *et al.* [21] provides a comprehensive survey of blockchain consensus protocols and an in-depth review of incentive mechanism designs. The paper starts with a layered view of the blockchain network including the consensus part followed by a general discussion on the compatibility between consensus protocol and incentive mechanism. Then detailed consensus schemes are introduced, including Nakamoto consensus, proof-of-concept consensus schemes, and virtual mining techniques such as proof of stake (PoS) along with the usability of trusted hardware. Hybridization between PoX and classical BFT protocols is also discussed. This survey features a game-theoretical characterization of Nakamoto consensus' incentive mechanism and its influence on system fairness and decentralization. The efficiency-scalability trade-off is also explored. Despite of having rich details on consensus techniques and insights into blockchain protocol design, this survey does not provide a concise abstraction that captures different functional components of a consensus protocol or a cohesive characterization on fault-tolerant distributed consensus primitives that can be used for blockchain consensus.

Xiao *et al.* [22] provides a succinct tutorial on distributed consensus protocols, from classical BFT protocols to the Nakamoto consensus protocol as well as recent breakthroughs in blockchain consensus. Specially this tutorial provides an abstraction of consensus goals for permissionless blockchains following the paradigm of classical BFT consensus. The authors also remark that the network model and trust model should be jointly considered when designing blockchain consensus protocols for practical applications. However, this tutorial skips details on each consensus protocols and does not clarify how different components of a blockchain consensus protocol contribute to system performance and security.

Belotti *et al.* [23] provides a *vademecum* on blockchain technology including development history, transaction and ledger structure, blockchain system abstraction, consensus mechanisms, and a detailed guide on when and how to use which blockchain technology. The consensus mechanisms covered include PoW, PoS, BFT algorithms, and hybrid BFT-based algorithms. Notably, this paper also gives a quantitative comparison of consensus mechanisms with respect to fault tolerance, node scalability, throughput, and transaction latency. Though with great details on building and managing a blockchain system, this vademecum can be perfected with an abstraction for each type of consensus mechanisms and a high-level tutorial on how to reach a compromise between different desired features.

III. Fault-Tolerant Distributed Consensus

The fault-tolerant (FT) distributed consensus problem has been extensively studied in distributed systems since the late 1970s and recently gained popularity in the blockchain community, especially for permissioned blockchains where every consensus participant reveals its identity. Generally, consensus in a distributed system represents a state that all participants agree on the same data values. Depending on the medium for message exchange, distributed systems are classified into two types: *message passing* and *shared memory* [24]. In this section we are interested in message passing systems because of their resemblance to contemporary blockchain systems, wherein distributed consensus on a single network history is reached through peer-to-peer communication. We will use the terms process/node/server interchangeably, as they all refer to an individual participant of distributed consensus.

A. System Model

1) Distributed System and Task: We consider a distributed system that consists of *N* independent processes. Each process p_i begins with an individual initial value x_i and communicates with others to update this value. Each local value can be used for a certain task, such as computation or just storage. If the processes are required to perform the same task, consensus on a single value is required before they proceed to the task.

2) Process Failure: A process suffers a *crash failure* if it abruptly stops working without resuming. The common causes of a crash failure include power shutdown, software error, and DoS attack. A *Byzantine failure*, however, is much severer in that the process can act arbitrarily while appearing normal. It can send contradicting messages to other processes in hope of sabotaging the consensus. "Byzantine" was coined by Lamport *et al.* [25] in 1982 when describing the *Byzantine Generals Problem*, an allegorical case for single-value consensus among distributed processes. The common cause of a Byzantine failure is adversarial influence, such as malware injection and physical device capture. Multiple Byzantine processes may collude to deal more damage.

3) Network Synchrony: Network synchrony defines the level of coordination among all processes. Three levels of synchrony, namely *synchronous*, *partially synchronous*, and *asynchronous*, are often assumed in the literature [22], [26].

- In a *synchronous* network, operations of processes are coordinated in rounds with clear time constraints. In each

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

round, all processes perform the same type of operations. This can be achieved by a centralized clock synchronization service and good network connectivity. Practically, a network is considered synchronous if message delivery is guaranteed within a fixed delay Δ, for which the network is also called Δ-synchronous.

- In a *partially synchronous* network, operations of processes are loosely coordinated in a way that message delivery is guaranteed but with uncertain amount of delays. Within the scope of partial synchrony, *weak synchrony* requires message delay not grow faster than the elapsing time indefinitely [27], while *eventual synchrony* ensures Δ-synchrony only after some unknown instant [28]. In either case, operations of the networked processes can still follow that of a synchronous network if the time horizon is long enough.
- In an *asynchronous* network, operations of processes are hardly coordinated. There is no delay guarantee on a message except for its eventual delivery. And the coordination of processes (if there is any) is solely driven by the message delivery events. This is often caused by the absence of clock synchronization (thus no notion of shared time) or the dominance of a mighty adversary over all communication channels.

It has been shown by Fischer *et al.* [29] that under the asynchronous case, consensus cannot be guaranteed with even a single crash failure. This is commonly known as the FLP impossibility, for the authors' namesake. However, this impossibility can be practically circumvented using randomized decision making and a relaxed termination property, as we will show in Section III-E.

B. Byzantine Fault Tolerant Consensus

We call a consensus protocol *crash fault tolerant* (CFT) or *Byzantine fault tolerant* (BFT) if it can tolerate a certain amount of crash or Byzantine process failures while keeping normal functioning. Because of the inclusive relationship between a crash failure and Byzantine failure, a BFT consensus protocol is naturally CFT. BFT consensus is defined by the following four requirements [22], [26], [28]:

- *Termination:* Every non-faulty process decides an output.
- *Agreement:* Every non-faulty process eventually decides the same output $\hat{y}$.
- *Validity:* If every process begins with the same input $\hat{x}$, then $\hat{y} = \hat{x}$.
- *Integrity:* Every non-faulty process' decision and the consensus value $\hat{y}$ must have been proposed by some non-faulty process.

The four requirements provide a general target for distributed consensus protocols. For any consensus protocol to attain these BFT requirements, the underlying distributed network should satisfy the following condition: $N \geq 3f + 1$ where f is the number of Byzantine processes. This fundamental result was first proved by Pease *et al.* [30] in 1980 and later adapted to the BFT consensus framework. The proof involves induction from $N = 3$ and partitioning all processes into three equal-sized groups, with one containing the faulty ones. Interested readers are referred to [26], [30] for detailed proofs.

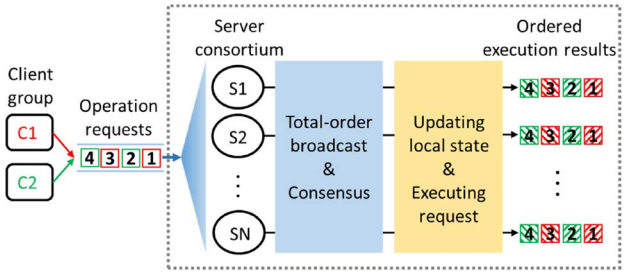


Fig. 1. A high-level illustration of SMR-based distributed computing that serves four operation requests from two clients.

C. Consensus in Distributed Computing

Consensus in distributed computing is a more sophisticated realization of the aforementioned distributed system. In a typical distributed computing system, one or more clients issue operation requests to the server consortium, which provides timely and correct computing service in response to the requests despite some of servers may fail. Here the correctness requirement is two-fold: correct execution results for all requests and correct ordering of them. According to Alpern and Schneider's work on liveness definition [31] in 1985, the correctness of consensus can be formulated into two requirements: *safety*—every server correctly executes the same sequence of requests, and *liveness*—all requests should be served.

To fulfill these requirements even in the presence of faulty servers, server replication schemes especially *state machine replication* (SMR) are often heralded as the de facto solution. SMR, originated from Lamport's early works on clock synchronization in distributed systems [32], [33], was formally presented by Schneider [34] in 1990. Setting in the client-server framework, SMR sets the following requirements:

1) All servers start with the same initial state;
2) Total-order broadcast: All servers receive the same sequence of requests as how they were generated from clients;
3) All servers receiving the same request shall output the same execution result and end up in the same state.

Total-order broadcast is also known as *atomic broadcast* (ABC) [35], which is in contrast to the *reliable broadcast* (RBC) [36] primitive. The latter only requires all servers receive the same requests without enforcing the order. It is shown in [37], [38] that atomic broadcast and distributed consensus are equivalent problems.

A high-level diagram of SMR-based distributed computing is illustrated in Fig. 1. The N-server consortium accepts operation requests from clients. Servers confirm each other's state before reaching consensus and executing requests. In many cases, especially randomized consensus protocols, there can be an alternating procedure of total-order broadcast and local state update until a certain consensus target is met, before moving on to execution. In practice, SMR is often implemented in a leader-based fashion. A primary server (say S1 in Fig. 1)

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

receives client requests and starts the broadcast procedure so that the other $N-1$ replica servers receive the same requests and update their local states to that of the primary.

In the rest of this section we summarize several well-known consensus protocols (some are based on SMR) designed under different network synchrony assumptions.

D. Consensus Protocols for Partially Synchronous Network

The ground-breaking work by Dwork *et al.* [28] in 1988 laid the theoretical foundation of partially synchronous consensus. By dissecting the consensus objective into *termination* and *safey*, the authors were able to formally prove the feasibility of four consensus goals, including CFT, omission-tolerance, BFT, unauthenticated BFT, under the Δ-synchrony/eventual synchrony condition. Notably, this work has inspired numerous proposals for partially synchronous consensus schemes, including the later known PBFT.

1) DLS Protocol: The same paper [28] also proposes a prototype consensus protocol (called DLS for authors' namesake) featuring a broadcast primitive for each consensus cycle. Specifically, the broadcast primitive is started by an arbitrary process p and consists of two initial rounds and subsequent iterative rounds. Through the message exchange in each round, the iterative procedure eventually drives the processes to reach agreement on a common value (either the one proposed by p or a default value). At message complexity $O(N^2)$ (N is the number of processes), the broadcast primitive essentially enables the DLS protocol to tolerate f Byzantine processes if $N \geq 3f+1$. The cryptocurrency Tendermint uses an adapted version of DLS for block finalization.

2) Viewstamped Replication (VR): Proposed by Oki and Liskov [39] in 1988, viewstamped replication is a server replication scheme for handling server crashes. It was later extended into a consensus protocol by Liskov and Cowling [40] in 2012, which we will refer to as VR. VR is a SMR scheme designed in the client-server framework and consists of three sub-protocols: 1) Normal-operation, 2) View-change, 3) Recovery. The primary server receives a client request and starts the normal operation, as is shown in Fig. 2(a). In the case of a crash failure of the primary, the View-change protocol is triggered at every replica per the timeout of the *Prepare* message. They broadcast *View-change* messages to each other and count the receptions. After receiving *View-change* messages from more than half of the replicas, the next-in-line replica becomes the new primary and informs the others to resume the normal operation. The Recovery protocol is used by any server to recover from a crash. VR can tolerate f crashed replicas if the network population $N \geq 2f+1$. However it does not tolerate any Byzantine failure, because the replicas simply follow the instructions from the primary without mutual state confirmation nor communication with the client. On the up side, this makes VR efficient, with $O(N)$ message complexity.

3) Paxos: Paxos is a SMR scheme proposed by Lamport [41] in 1989 that imitates the ancient Paxos part-time parliament and later elaborated in 2001 [42]. It was designed specifically for fault tolerant consensus while bearing many similarities to VR. Paxos classifies nodes into three roles: proposers, acceptors, and learners. A proposer suggests a value in the beginning and the system goal is to make acceptors agree on a single value, and learners learn this value from acceptors. In the client-server scenario depicted in Fig. 2(b), the client is the learner, the primary is the proposer, and the replicas are acceptors. After updating to the same state, all servers execute the request and send it to the client who then chooses the majority result. When the proposer suffers a crash failure, the acceptors elects a new leader through a similar propose-accept procedure. Akin to VR, Paxos can tolerate f crashed acceptors when $N \geq 2f+1$, but no Byzantine failures. Because of the mutual messaging during the accept phase, the message complexity of Paxos is $O(N^2)$.

Embarking from its original design, Paxos has grown into a family of consensus protocols, including multi-Paxos, cheap-Paxos, and fast-Paxos, each features a specific goal. Raft, a SMR consensus protocol developed by Ongaro and Ousterhout [43] in 2014 and popular in the blockchain community, is based off Paxos but with a more understandable design.

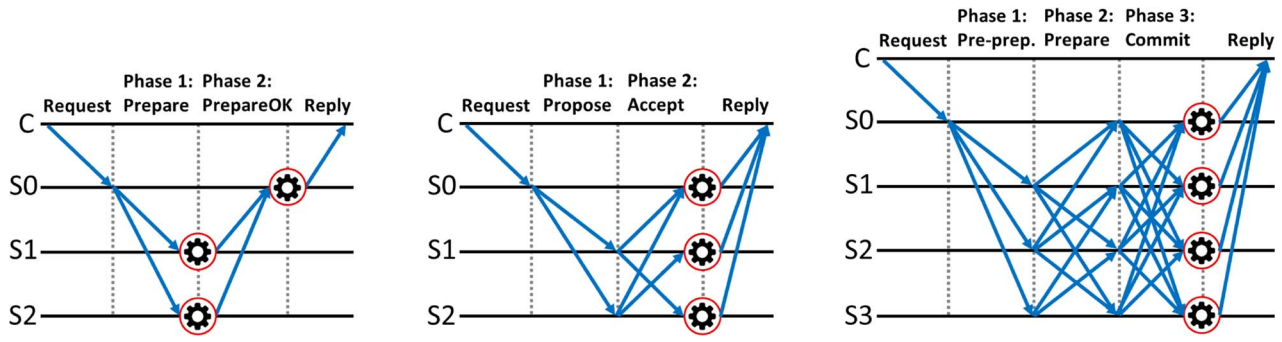
4) Practical Byzantine Fault Tolerance (PBFT): Developed by Castro and Liskov [27] in 1999, PBFT is the first SMR-based BFT consensus protocol that has gained wide recognition for practicality. It has become almost synonymous to BFT consensus in the blockchain community. PBFT originated from the VR framework and took inspiration from Paxos. PBFT consists of three sub-protocols: 1) Normal-operation, 2) Checkpoint, 3) View-change.

The Normal-operation protocol is shown in Algorithm 1 and Fig. 2(c). Ideally, all results replied to the client should be the same; otherwise the client chooses the majority result. The Checkpoint protocol serves as a logging tool that keeps a sliding window (of which the lower bound is the stable checkpoint) to track active operation requests. The latest stable checkpoint is used for safely discarding older requests in

Algorithm 1: PBFT (Normal-Operation Protocol)

```
/* Request                                          */
Client sends an operation request to the primary;
/* Phase 1: Pre-prepare                             */
The primary relays this request to replicas via
  Pre-prepare messages;
Replicas record the request and update local states;
/* Phase 2: Prepare                                 */
Replicas send Prepare messages to all servers (replicas
  and the primary);
Once receiving ≥ 2f + 1 Prepare messages, every server
  updates local state and is ready to commit;
/* Phase 3: Commit                                  */
Servers send Commit messages to each other;
Once receiving ≥ 2f + 1 Commit messages, every server
  starts to execute the client request and then updates
  local state;
/* Reply                                            */
Every server replies its result to the client.
```

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.



(a) VR (b) Paxos (c) PBFT

Fig. 2. Messaging diagram during the normal operation of three SMR protocols. C is the client. S0 is the primary server (leader) who receives requests from the client and starts the consensus. S1+ are replica servers. Every server updates local state after receiving a message. Circled gear icon represents request execution. VR and Paxos can tolerate one crash failure when $N = 3$. PBFT can tolerate one Byzantine failure when $N = 4$.

the operation log and facilitating the view change protocol. In the case of a primary failure, the View-change protocol is triggered at every replica that detects the timeout of the primary's message. They oust the incumbent primary and broadcast view-change messages to each other and count receptions. After receiving *View-change* messages from $2f$ peers, the next-in-line replica becomes the new primary and informs the rest to resume the normal operation.

The message complexity of PBFT normal operation is $O(N^2)$ because of the mutual messaging in *Prepare* and *Commit* phase. As for the fault tolerance, since a server needs to receive more than $2f + 1$ *Prepare* (*Commit*) messages in Prepare (Commit) phase before proceeding to the next action, there will be at least $2f + 1$ (as $N \geq 3f + 1$) honest servers in the same state after *Commit* phase and producing the same result; the f Byzantine servers are not able to sway the majority consensus. Therefore PBFT can tolerate f Byzantine replicas when the server population $N \geq 3f + 1$, which is in accordance to the fundamental 1/3 BFT threshold. Interested readers are referred to [22], [27] for detailed proofs.

PBFT has inspired numerous BFT consensus protocols with enhanced security and performance. Well-known proposals include Quorum/Update (QU) [44], Hybrid Quorum (HQ) [45], Zyzzyva (using speculative execution) [46], FaB [47], Spinning [48], Robust BFT SMR [49], and Aliph [50]. Interested readers are referred to Bessoni's tutorial [51] for an overview of these protocols.

E. Consensus Protocols for Asynchronous Network

For distributed systems that are predominantly built upon wired communication and reliable transport-layer protocols, partial synchrony is a practical assumption. However in scenarios such as mobile ad hoc network (MANET) and delay tolerant networks (DTN), the network is considered of near-to-none synchrony. As is proved by the FLP impossibility [29], consensus can not be guaranteed in a fully asynchronous network with even one crash failure. Moreover, unreliable communication links have an equivalent effect of a Byzantine scheduler. Nonetheless, this impossibility result can be practically circumvented by two primitives: *probabilistic termination* and *randomization*.

First of all, according to [52] the termination property presented in Section III can be subdivided into two classes:

- *Deterministic termination:* Every non-faulty process decides an output by round r, a predetermined parameter.
- *Probabilistic termination:* The probability that a non-faulty process is undecided after r rounds approaches zero as r grows to infinity.

For synchronous or partially synchronous networks where message delay and round period are bounded, protocols like PBFT can exploit a timeout mechanism to detect anomaly of the primary, which makes deterministic termination an achievable goal. For asynchronous networks where messages delivery has no timing guarantee, the consensus process can only be driven by the message delivery events themselves. This limitation demands probabilistic termination. To realize probabilistic termination, randomization (simultaneously proposed by Ben-Or [53] and Rabin [54] in 1983) can be instantiated in the consensus protocol. The basic idea is that a process makes a random choice when there are not enough trusted messages received for making a final decision.

Next we introduce four primitives/protocols that aim to solve asynchronous BFT consensus. Though in different contexts, they all feature probabilistic termination and make use of randomization.

1) Bracha's RBC and Asynchronous Consensus Protocol: Bracha [55] proposed the pioneering reliable broadcast (RBC) primitive and an asynchronous consensus protocol in 1984 to solve the Byzantine Generals Problem [25], in which all non-faulty processes should eventually make the same binary decision. Bracha's RBC guarantees that non-faulty processes will never accept contradicting messages from any process and forces the faulty ones to output either nothing (mimicking the crash failure) or the correct value. Bracha's asynchronous consensus protocol, adapted from Ben-Or's 1983 work [53], runs by phases and each phase contains three RBC rounds for inter-process value exchange. We show the round-3 of each phase, which contains the randomization step. After receiving at least $N - f$ value messages (f is the presumed Byzantine population), a process P_i does:

1) If receiving a value v from more than $2f$ peers, decide v;

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Ben-Or's ACS Protocol (at Player P_i)

```
/* Phase 1: Reliable Broadcast */
Start RBC_i to propose my input x_i to the network;
Participate in other RBC instances;
/* Phase 2: Asynchronous BA */
while round ≤ MaxRound do
    if receiving x_j from RBC_j then
        Join ABA_j with input 1;
    end
    if completion of N − f ABA instances then
        Join other BA instances with input 0;
    end
    if completion of all N ABA instances then
        ComSubset = {x_k | ABA_k outputs 1};
        return ComSubset;
    end
end
```

2) Else if receiving a value v from more than f peers, hold v as proposal value and go to the next phase;
3) Else, toss a coin (1/2 chance for 0 or 1) for the proposal value and go to the next phase.

When enough phases pass, the executions of step 2 and step 3 of RBC round-3 at all non-faulty processes will gradually filter out the influence of contradicting messages and eventually make the correct decision via step 1. Note this convergence only happens if $N \geq 3f + 1$, which is the fundamental bound of BFT consensus.

In terms of performance, the message complexity of RBC is $O(N^2)$ in each round and the expected number of rounds to reach consensus is $O(2^N)$ if $f = O(N)$, which gives a total message complexity of $O(N^2 2^N)$. If $f = O(\sqrt{N})$ (the benign case), it is shown in [53], [55] that the expected number of rounds to reach consensus for the randomized protocol is a constant, yielding a total message complexity of $O(N^2)$.

2) Ben-Or's ACS Protocol for MPC: Agreement on a common subset (ACS) was used by Ben-Or *et al.* [56] in 1994 as a consensus primitive for secure and efficient multi-party computation (MPC) under asynchronous setting. In a network of N players, each player holds a private input x_i that was acquired secretly. The goal of MPC is to let the players collectively compute a function $\mathscr{F}(x_1, \ldots, x_N)$ and obtain the same result. Assuming f players can be faulty, the ACS primitive requires the players to agree on a common subset *ComSubset* of at least $N - f$ honest inputs, which are then used for computing $\mathscr{F}(\cdot)$.

Ben-Or's ACS protocol builds on two primitives: RBC and binary asynchronous Byzantine agreement (ABA) which allows players to agree on the value of a single bit. Bracha's RBC [55] and Canetti and Rabin Fast ABA [57] are suggested respectively in [56] and used as black-boxes. Algorithm 2 shows the ACS protocol at each player. In the end, there will be at least $N - f$ completed ABA instances with output 1, yielding a $\mathscr{F}$-computable *ComSubset*.

Because both Bracha's RBC and Canetti's ABA can tolerate f Byzantine players when $N \geq 3f + 1$, the same fault tolerance result is inherited by Ben-Or's ACS protocol. For complexity analysis, Bracha's RBC (in the benign case) and Canetti's ABA have message complexity of $O(N^2)$ and $O(N^3)$ [57] respectively, and all ABA instances end in constant rounds. As a result, Ben-Or's ACS protocol has a bit-denominated communication complexity of $O(mN^2 + N^3)$ at each player, where m is the maximum bit-size of any input.

As we will see next, ACS can be conveniently adapted to asynchronous BFT consensus for blockchain systems, by substituting inputs with transaction sets.

3) HoneyBadgerBFT: Proposed by Miller *et al.* [16] in 2016, HoneyBadgerBFT is the first asynchronous BFT consensus protocol specifically designed for blockchain. It essentially realizes atomic broadcast: N players with different sets of transactions work to agree on a common set of sorted transactions that will be included in a block.

Though using the multi-value Byzantine agreement primitive (MVBA) from Cachin *et al.* [58] as the benchmark, HoneyBadgerBFT actually follows Ben-Or's ACS construction [56] for better communication efficiency. HoneyBadgerBFT's ACS cherry-picks the design of its subcomponents: Cachin and Tessaro's erasure-coded RBC [59] and Mostéfaouil *et al.*'s common-coin based ABA [60]. They together incur $O(mN + N^2 \log N)$ communication complexity at each player. To prevent an adversary from censoring particular transactions, threshold public key encryption (TPKE) [61] is used before ACS so that consensus is performed on ciphertexts. In the decryption phase when a player receives enough shares from peers (generated by *TPKE.DecShare*) that exceed a threshold, it proceeds to the actual decryption task (*TPKE.Dec*) and sorts the transactions. The communication complexity of the decryption process is $O(N^2)$. Fig. 3 illustrates the workflow of these components for one block cycle.

HoneyBadgerBFT processes transactions in batches. Let B be the predefined batch size, denoting the maximum number of transactions that a block may enclose. For every block cycle, each player proposes a set of B/N transactions, which are randomly chosen from recorded transactions. This is to ensure transaction sets proposed by different players are mostly disjoint so as to maximize blockchain throughput. Assuming the average bit-size of a transaction set $m := \frac{|t|B}{N} \gg N$ where $|t|$ is the average transaction bit-size. Then the protocol's communication overhead will be dominated by the RBC, yielding overall communication complexity of $O(|t|B)$ at one player, or $O(|t|N)$ for one transaction.

Compared to popular partially synchronous consensus protocols such as PBFT, HoneyBadgerBFT has a higher cryptography overhead but features two advantages. First, as an asynchronous protocol HoneyBadgerBFT does not rely on a timeout mechanism for detecting malfunctioning players. This makes HoneyBadgerBFT less sensitive to unpredictable network delays that might stall consensus. Second, HoneyBadgerBFT does not need a leader rotation scheme. In PBFT every round of consensus is started by a leader (the primary), while in HoneyBadgerBFT every node starts its own broadcast and Byzantine agreement instance for proposed transactions; the concurrent execution of these instances

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

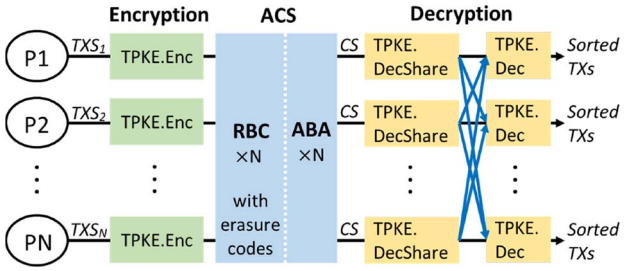


Fig. 3. HoneyBadgerBFT workflow. TXS_i is the set of transactions proposed by player P_i. *CS* is the common subset output of ACS, comprising of at least $N - f$ encrypted transactions. The decryption process outputs sorted transactions that will be finalized in the block.

effectively saves the need of a leader. As a result, the bandwidth of any individual leader will not become a bottleneck of the entire network's transaction capacity. Currently the blockchain initiative POA Network [11] is considering to adopt HoneyBadgerBFT.

On the other hand, due to HoneyBadgerBFT's asynchronous design philosophy that consensus progress is driven by message deliveries, transaction confirmation latency is externally influenced and uncontrollable. This leads HoneyBadgerBFT to overly emphasize high transaction throughput and decentralization. In various applications such as industrial control and supply chain management, low transaction latency is often times a more important metric than throughput.

In response to the said inflexibility of HoneyBadgerBFT, Duan *et al.* [17] proposed BEAT in 2018, which is a collection of five asynchronous BFT protocols based off HoneyBadgerBFT but with carefully picked components that are optimized for different objectives. Among the five constituent protocols, the baseline BEAT0 uses a more efficient threshold encryption scheme [62] and outperforms HoneyBadgerBFT in throughput, latency and access overhead. BEAT1 and BEAT2 adopt a more efficient broadcast scheme, Bracha's RBC [52], and are optimized for transaction latency. BEAT3 is optimized for throughput and storage and bandwidth saving while BEAT4 further reduces the bandwidth usage for clients that read particular stored transactions. Interested readers are referred to the BEAT paper [17] for detailed discussion on the authors' design choices.

F. Blockchain Compatibility of Classical BFT-SMR Protocols

In a blockchain network, every consensus participant can validate transactions and propose new blocks. For BFT-SMR consensus protocols that rely on a dedicated primary server to receive client requests and start the consensus, the following adaptation is needed: allowing all servers to act as a primary to propose transactions/blocks and reaching consensus on the finality of multiple transactions/blocks concurrently. For example, Casper FFG [14], a BFT-style blockchain protocol, allows every eligible participant to propose a block during a checkpoint cycle. The network finalizes only one block out of multiple proposed blocks for each checkpoint.

For blockchain networks with a complex application layer such as smart contract, transaction execution often incurs significant computation. While in popular BFT-SMR schemes such as PBFT execution is integrated into the consensus process. An early work by Yin *et al.* [63] presents an alternative BFT-SMR framework that separates consensus (i.e., agreement on execution order) from execution, as the latter conveniently requires only an honest majority of execution nodes instead of an honest two-thirds of consensus nodes required by the former. This separation scheme is adopted by Zyzzyva [46] and TenderMint [12] wherein a small group of nodes are dedicated to the consensus task. Hyperledger Fabric [15] further separates the consensus task into ordering service and validation service for better modularity.

From the performance perspective, BFT protocols are notorious for their limited scalability in network size. Epitomized by PBFT, the message complexity of partially synchronous BFT protocols grows quadratically with the network size *N*. This means that given a fixed network bandwidth at each node, a growing network size leads to exploding communication overhead and diminishing transaction capacity. According to the performance evaluation in [16], PBFT achieves a maximum throughput of 16,000 TPS when $N = 8$; this figure drops to around 3,000 when $N = 64$. On the other hand, for asynchronous protocols like HoneyBadgerBFT where erasure coding and threshold encryption are used to reduce communication complexity and enhance security, the extensive use of cryptography also brings non-negligible computation overhead, adding to local processing delays. On the bright side, a typical BFT protocol achieves *deterministic finality*, which is also known as *forward security* [64] in that a settled transaction will never be altered. As we will discuss in Section V, this allows BFT protocols to take advantage of shorter block intervals and attain high transaction throughput.

Other blockchain compatibility challenges for BFT-SMR protocols include: 1) allowing nodes to join and leave flexibly without interrupting consensus while countering Sybil attacks; 2) adapting to real-world peer-to-peer networks that are sparsely connected. In later sections we will revisit these issues for blockchain protocols that incorporate BFT consensus.

IV. An Overview of Blockchain Consensus

Compared to traditional distributed computing with a clear client-server model, a blockchain network allows every participant to be both a client (to issue transactions) and a server (to validate and finalize transactions). The underlying ledger data structure, the blockchain, is the consensus target and consists of chronologically ordered and hash-chained blocks. Each block contains a bundle of valid transactions and transactions across the blockchain should be consistent with each other (i.e., no double-/over-spending or misappropriation). Meanwhile, a blockchain system is often associated with a financial application and bears the responsibility of transaction processing and clearing. As a result, the responsibility of a blockchain consensus protocol is further-reaching than traditional distributed consensus protocols. In this section we provide a background of the blockchain network and data structure, introduce the blockchain consensus goal adapted from the BFT consensus paradigm and the five-component framework that we use to analyze blockchain consensus protocols.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

A. Blockchain Infrastructure

Network: The foundational infrastructure of blockchain, as is adopted by most public cryptocurrencies and distributed ledger systems, is a peer-to-peer overlay network on top of the Internet. Every node (or peer) in the network operates autonomously with respect to the same set of rules that cover peering protocol, consensus protocol, transaction processing, ledger management, and in some cases a wire protocol for transport-layer communication [65], [66].

Depending on the control of network participation, blockchain networks generally fall into two categories: permissionless and permissioned.

- A *permissionless* blockchain allows for free join and leave without any authorization, as long as the node holds a valid pseudonym (account address) and is able to send, receive, and validate transactions and blocks by common rules. Permissionless blockchain is also known as public blockchain for that there is usually one such blockchain network instance on a global scale which is subject to public governance. Specifically, anyone can participate in blockchain consensus, though one's voting power is typically proportional to its possession of network resources, such as computation power, token wealth, storage space, etc. The operational environment of permissionless blockchain is often assumed as large-scale and zero-trust, which often cautions the community against increasing transaction processing capacity or using more efficient consensus schemes [22].
- A *permissioned* blockchain requires participants to be authorized first and then participate in network operation with revealed identity. The network governance and consensus body can be either the subsidiaries of a single private entity or a consortium of entities [19]. Compared to permissionless blockchain, the identity-revealing requirement and more effective network governance of permissioned blockchain make it ideal for internal or multi-party business applications. Meanwhile, the limited size of a permissioned blockchain's consensus body allows for the deployment of more efficient consensus protocols that achieve higher transaction capacity [22], [67].

Table I summarizes the major differences between permissionless and permissioned blockchain in nine aspects.

Beneath the blockchain peer-to-peer network lies the basic infrastructure of the Internet. Thanks to the transport layer protocols (especially the retransmission mechanism), message delivery is considered guaranteed, while the message delay may vary but most likely will not grow longer as time elapses (weak synchrony) or remains within a certain bound (Δ-synchrony). Therefore we often consider a practical blockchain network partially synchronous, just like most distributed networks overlaying on the Internet. This allows the consensus protocol to take advantage of the timing services of the Internet. For example, in Bitcoin the partial synchrony assumption is echoed by its usage of local timestamps for loose chronological ordering, showing time consciousness. For the blockchain networks that reside on an ad hoc infrastructure not based on the Internet, the message transmission is subject to unexpected network delays, which gives rise to asynchronous consensus protocols such as HoneyBadgerBFT, as we discussed in Section III.

TABLE I
A COMPARISON OF PERMISSIONLESS AND PERMISSIONED BLOCKCHAIN

	Permissionless blockchain	**Permissioned blockchain**
Governance	Public	Private / Consortium
Participation	Free join and leave	Authorized
Node identity	Pseudonymous	Revealed
Transparency	Open	Closed / Open
Network size	Large (thousands or more)	Small (tens∼hundreds)
Network connectivity	Low	High (oft. fully-connected)
Network synchrony	Asynchronous / partially synchronous	Partially synchronous / synchronous
Transaction capacity (tps)	Low (oft. sub-ten∼tens)	High (oft. thousands)
Application examples	Cryptocurrency, smart contract, public record, DApp	Inter-bank clearing, business contract, supply chain

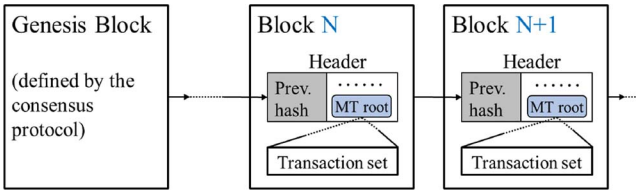


Fig. 4. Blockchain data structure. Blocks are sequentially chained together via hash pointers. The Merkle tree root (MT root) is a digest of all transactions included in a block.

Transaction: A blockchain transaction can be regarded as a public static data record showing the token value redistribution between sender and receiver [21]. Take Bitcoin as an example, a transaction transfers token ownership from the sender account to the receiver account(s). It specifies a list of inputs and a list of outputs, with each input claiming a previous unspent transaction output (UXTO) that belongs to the sender, who needs to attach its signature to the inputs to justify the claim. Each output specifies how many tokens go to which receiver and the total token value of the outputs is equal to the UXTOs claimed by the inputs. Therefore, we can always recover the ownership records of any specific token by tracing back the signatures along the chain of transactions. The token balance of an account equals to the summed UXTOs that belong to the account.

Blockchain Data Structure: Blockchain is the underlying data structure for transaction ledger keeping. It is also the consensus target of the network. The basic structure of blockchain is illustrated in Fig. 4. Every block encloses a set of transactions that should be valid and clear of double spending. As was pioneered by Bitcoin, the transactions are often organized in a Merkle tree. Merkle tree is a data structure widely used for data storage and efficient data integrity check [68]. In blockchain, every block contains one Merkle tree in which each leaf node is labeled with a transaction hash. The Merkle

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

TABLE II
FIVE COMPONENTS OF A BLOCKCHAIN CONSENSUS PROTOCOL

Component	Purpose	Counterpart in traditional SMR consensus protocols	Available options
Block proposal	Generating blocks and attaching essential generation proofs (for Sybil attack resistance).	Clients issuing operation requests and the primary server starting the consensus.	Proof of work (PoW), proof of stake (PoS), proof of authority (PoA), proof of retrievability (PoR), proof of elapsed time (PoET), round robin, committee-based, etc.
Information propagation	Disseminating blocks and transactions across the network.	Reliable broadcast of operation requests.	Advertisement-based gossiping, block header soliciting, unsolicited block push (broadcast), relay network (for mining pools), etc.
Block validation	Checking blocks for generation proofs and validity of enclosed transactions.	Signature check and execution of operation requests.	Proof checking (for proof-of-X block proposal), digital signature & eligibility checking (for committee-based block proposal), etc.
Block finalization	Reaching agreement on the acceptance of validated blocks.	Servers reaching an agreement on current state, executing requests and logging the result.	Longest-chain rule, GHOST rule, BFT and other Byzantine agreements, checkpointing, etc.
Incentive mechanism	Promoting honest participation and creating network tokens.	N/A.	Network token rewards (block rewards, transaction fees), eligibility for issuing new transactions, etc.

tree root serves as a digest of the transaction set and is placed in the block header. The block header also contains a hash of the previous block (except in the genesis block) and other configuration information, which typically includes a timestamp and the blockchain state at block generation. The growing chain of blocks and the aforementioned transaction format essentially constitute the blockchain data structure used for the storage, serialization, and validation of new transactions which are continuously injected into the network.

Aside from recording the transaction history, the blockchain can also record auxiliary information used for other purposes. The locking and unlocking scripts associated with transaction inputs and outputs can be repurposed for constructing off-chain payment channels (e.g., Lightning Network [69]) and a global state machine that helps build smart contracts, which are the basis of many important applications such as supply chain management and decentralized autonomous organization. The block header may contain extra fields that facilitate system coordination. For example, Ethereum's proof-of-stake (PoS) scheme Casper FFG [70] utilizes smart contract to implement the staking process; Algorand [13] attaches a cryptographic proof to each new block to show the block proposer's eligibility to propose. As a result, the blockchain can hold the necessary control information usable by the consensus protocol. We will revisit these protocols in later sections.

B. Consensus Goal

The goal of a blockchain consensus protocol is to ensure that all participating nodes agree on a common network transaction history, which is serialized in the form of a blockchain. Based on the previous discussion on BFT consensus and the consensus goal abstraction provided in [22], we similarly define the following requirements for blockchain consensus:

- *Termination:* At every honest node, a new transaction is either discarded or accepted into the blockchain, within the content of a block.
- *Agreement:* Every new transaction and its holding block should be either accepted or discarded by all honest nodes. An accepted block should be assigned the same sequence number by every honest node.
- *Validity:* If every node receives a same valid transaction/block, it should be accepted into the blockchain.
- *Integrity:* At every honest node, all accepted transactions should be consistent with each other (no double spending). All accepted blocks should be correctly generated and hash-chained in chronological order.

The *termination* and *validity* requirements are similar to their counterparts in classical distributed consensus, as they represent the system's liveness. The *agreement* requirement is enhanced with total ordering, which represents the serialization of blocks and transactions. The *integrity* requirement dictates the correctness of the origin of transactions and blocks, fulfilling the promise of anti-double-spending and ledger tamper-proofing. These requirements can serve as the design principles of new blockchain protocols. For different application scenarios, they can be tailored or supplemented with more specification.

C. Components of Blockchain Consensus Protocol

Based on the discussion on consensus goal and our digest of the blockchain documentation corpus, we identify five key components of a blockchain consensus protocol:

- *Block Proposal:* Generating blocks and attaching generation proofs.
- *Information Propagation:* Disseminating blocks and transactions across network.
- *Block Validation:* Checking blocks for generation proofs and transaction validity.
- *Block Finalization:* Reaching agreement on the acceptance of validated blocks.
- *Incentive Mechanism:* Promoting honest participation and creating network tokens.

For each component we also specify its counterpart in traditional SMR consensus protocols and a list of available options in Table II. The available options are non-exhaustive, as many more are being developed at the time of writing. It is worth

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

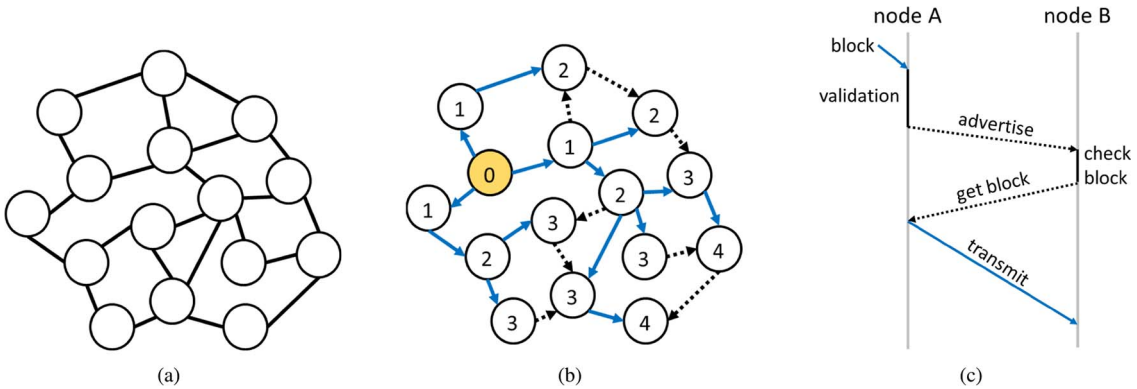


Fig. 5. A toy example of block propagation in the Bitcoin network. (a) The P2P network structure, an undirected graph. (b) The gossiping process. A solid blue arrow represents one-hop block propagation (advertise→get block→transmit), while a dotted black arrow represents only advertise. Number denotes the gossiping hop (0 for the block producer). (c) Block propagation in one hop.

noting that the incentive mechanism is unique to blockchain consensus and has no counterpart in traditional SMR consensus protocols. The reason is that traditional SMR protocols are purely designed for transaction processing and serialization within a preexisting network of participants, of which the continuous participation of honest parties is presumed. Meanwhile, a typical blockchain network allows for voluntary participation and often bears numerous real-world obligations. To this end, a fair and universal incentive mechanism is needed to encourage honest participation, so as to sustain the system's reliable operation. For large-scale permissionless blockchains, a robust incentive mechanism along with the block generation proofs help demoralize Sybil attackers.

Though the five components provide important functionalities for blockchain consensus, a new blockchain consensus protocol proposal may not cover all of them. For example, the incentive mechanism is indispensable to permissionless blockchain networks especially those carrying a financial responsibility; however for permissioned blockchains in which participation is sanctioned as a privilege (similar to a traditional distributed computing system), it is not a must-have. Interestingly, many new public blockchain initiatives have been fixating only on block proposal, while inheriting the other four components from the Nakamoto consensus protocol of Bitcoin. This is likely due to that Bitcoin's PoW-based block proposal attracts the most criticism for its limited scalability and inefficient energy use. For this reason, block proposal mechanism can be a good reference angle for a general classification of consensus protocols. In the remaining part we dedicate Section V to Nakamoto consensus and its variations that are built upon PoW, while Section VI is dedicated to four genres of PoS-based protocols. Detailed protocol composition is described when it comes to specific features.

V. The Nakamoto Consensus Protocol and Variations

The Nakamoto consensus protocol is the key innovation behind Bitcoin [1] and many other established cryptocurrency systems such as Ethereum [71] and Litecoin [72]. In this section we use Bitcoin as the application background to introduce the Nakamoto consensus protocol and summarize its drawbacks and vulnerabilities. We also introduce two well-known improvement proposals and four hybrid PoW-BFT protocols in the later part of this section.

A. Network Setting and Consensus Goal

In blockchain networks, block or transaction messages are propagated across the P2P network through gossiping. Fig. 5 shows an example of block propagation in the Bitcoin network. Specifically, the one-hop propagation adopts advertisement-based gossiping, as was first characterized in [73]. For each new block received and validated, a node advertises it to peers, who will request for this block if it extends their local blockchain. The gossiping process continues until every node in the network has this block.

Compared to the general consensus goal introduced in Section IV, Nakamoto enhances the termination requirement with the probabilistic finality specification:

- *Probabilistic finality:* For any honest node, every new block is either discarded or accepted into its local blockchain. An accepted block may still be discarded but with an exponentially diminishing probability as the blockchain continues to grow.

The probabilistic finality property echoes the probabilistic termination property for asynchronous consensus. As we will show later, because of this property the Nakamoto consensus protocol can only achieve eventual double-spending resistance in a decentralized network of pseudonymous participants.

B. The Nakamoto Consensus Protocol

In correspondence to the five components of a blockchain consensus protocol, the Nakamoto consensus protocol can be summarized by the following rules:

- *Proof of Work (PoW):* Block generation requires finding a preimage to a hash function so the hash result satisfies a difficulty target, which is dynamically adjusted to maintain an average block generation interval.
- *Gossiping Rule:* Any newly received or locally generated transaction or block should be immediately advertised and broadcast to peers.
- *Validation Rule:* A block or transaction needs to be validated before being broadcast to peers or appended to

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

the blockchain. The validation includes double-spending check on transactions and proof-of-work validity check on block header.

- *Longest-Chain Rule:* The longest chain represents network consensus, which should be accepted by any node who sees it. Mining should always extend the longest chain.
- *Block Rewards and Transaction Fees:* Generator of a block can claim a certain amount of new tokens plus fees collected from all enclosed transactions, in the form of a *coinbase* transaction to itself.

The hashing-intensive PoW mechanism is designed for mitigating Sybil attacks. Due to Bitcoin's permissionless and pseudonymous nature, Sybil attackers can obtain new identities or accounts with little effort. Hashing power, however, comes from real hardware investment and cannot be easily forged. The longest-chain rule implies that the stabilized prefix of the longest chain can act as a common reference of the network history, given that no one is authoritative in Bitcoin's decentralized network. Block rewards and transaction fees are used to incentivize miners to participate honestly and inject new coins into circulation.

To better illustrate how these rules harmonize with each other, we present an abstracted version of the Nakamoto protocol in Algorithm 3. During block generation, a higher mining difficulty demands more brute-force trials in order to find a fulfilling nonce. To ensure every block is sufficiently propagated before the next block comes out, the mining difficulty is adjusted every 2016 blocks so that the expected block interval remains a constant value (10 minutes in Bitcoin) no matter how the gross hashing power fluctuates.

Fork resolution: Ideally, the 10-minute block interval should be enough to ensure the thorough propagation of a new block so that no block of the same height is proposed. However due to the delay during the message propagation and the probabilistic nature of the hashing game, the possibility of two blocks of the same height being propagated concurrently in the network can not be ignored. This situation is called a "fork", detectable by any node. Correspondingly, the longest-chain rule provides the criterion for fork resolution. Assume a miner receives two valid blocks B_1^k, B_2^k of the same block height *k* sequentially, then a fork is detected by this miner. It chooses B_1^k (the first arrived) to continue and may encounter the following cases:

- *Case 1:* If receiving or successfully generating a block B_1^{k+1} confirming B_1^k, accept B_1^k, B_1^{k+1} and orphans B_2^k.
- *Case 2:* If receiving a block B_2^{k+1} confirming B_2^k, switch to B_2^{k+1} and accept B_2^k, then orphans B_1^k.
- *Case 3:* If simultaneously receiving two blocks B_1^{k+1} and B_2^{k+2} confirming respectively B_1^k and B_2^k, choose one to follow and continue until case 1 or 2 is met.

Wherein to "orphan" a block means to deny it into the main chain. Because of the randomized nature of PoW mining, the likelihood of encountering case 3 drops exponentially as time elapses, reflecting the probabilistic finality of Nakamoto consensus.

Algorithm 3: Nakamoto Consensus Protocol General Procedure

```
   /* Joining network                                  */
1  Join the network by connecting to known peers;
2  Start BlockGen();
   /* Main loop                                        */
3  while running do
4      if BlockGen() returns block then
5          Write block into blockchain;
6          Reset BlockGen() to the current blockchain;
           /* Gossiping rule                           */
7          Broadcast block to peers;
8      end
       /* Longest-chain/validation rule                */
9      if block received & is valid & extends the longest
        chain then
10         Write block into blockchain;
11         Reset BlockGen() to the current blockchain;
12         Relay block to peers;
13     end
14 end
   /* PoW-based block generation                       */
15 Function BlockGen():
16     Pack up transactions (including coinbase);
17     Prepare a block header context 𝒞 containing the
        transaction Merkle tree root, hash of the last block
        in the longest chain, timestamp, and other essential
        information reflecting blockchain status;
       /* PoW hashing puzzle                           */
18     Find a nonce that satisfies the following condition:
                        Hash(𝒞|nonce) < target
        wherein more preceding zero bits in target indicate
        a higher mining difficulty;
19     return new block;
20 end
```

Security Analysis: In contrast to the classical distributed computing system whose fault tolerance capability is characterized by the number of faulty nodes the system can tolerate, fault tolerance of Nakamoto consensus is by characterized by percentage of adversarial hashing power the system can tolerate. It is proved by Garay *et al.* [74] that if the network synchronizes faster than the PoW-based block proposing rate, an honest majority among the equally-potent (in hashing power and bandwidth) miners can guarantee the consensus on an ever-growing prefix of the blockchain. The prefix represents the probabilistically stable part of the blockchain. As long as less than 50% of total hashing power is maliciously controlled, the blocks produced by honest miners are timely propagated, the main chain contributed by the honest majority can eventually outgrow any malicious branch.

From the perspective of classical distributed consensus, Nakamoto consensus cleverly circumvents the fundamental 1/3 BFT bound by adopting probabilistic finality. In classical BFT

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

consensus if more than 1/3 of population are malicious, the honest nodes will end up deciding conflicting values, leading to consensus failure. In Nakamoto consensus, however, conflicting decisions are allowed temporarily in the form of blockchain forks, as long as they will be eventually trimmed out by continuing effort of the honest majority. Therefore, the 1/3 BFT bound is not applicable to Nakamoto consensus or other blockchain consensus protocols designed for probabilistic finality. Readers are referred to Abraham and Malkhi [75] for an interesting discussion on the correspondence between Nakamoto consensus and classical BFT-SMR framework.

As for double-spending resistance, assuming the adversary controls α fraction of the total hashing power and wishes to double-spend an output which is *m* blocks old, it needs to redo the PoW mining all the way from *m* blocks behind and grow a malicious chain fast enough to overtake the incumbent main chain. The probability of this adversarial catch-up is $(\frac{\alpha}{1-\alpha})^m$ if $\alpha < 50\%$, which drops exponentially as *m* increases, reflecting probabilistic finality. This probability equals to 1 if $\alpha \geq 50\%$. As a result, the 50% threshold is the safeguard behind Bitcoin's probabilistic finality, as well as resistance to double-spending and transaction history tampering.

C. Drawbacks and Vulnerabilities of Nakamoto Consensus

1) Tight Tradeoff Between Performance and Security: The Nakamoto consensus is widely criticized for its low transaction throughput. For instance, Bitcoin can process up to 7 TPS meanwhile the VISA payment network processes 2500 TPS on average [76]. The limited performance of Nakamoto consensus protocol follows from the security implication of its probabilistic finality and two protocol parameters: block interval and block size. As we discussed previously, the 10-minute block interval ensures every new block is sufficiently propagated before a new block is mined. Reducing the block interval increases the transaction capacity, but leaves new blocks insufficiently propagated and causes more fork incidents, undermining the security of the main chain. Note that although any fork can be resolved given enough time, the higher the fork rate, the larger the portion of honest mining power is wasted, which enables a double-spending attacker to overthrow the main chain with less than 50% mining power (estimated chance of 49.1% by Decker and Wattenhofer [73] in 2013). On the other hand, increasing the block size (currently 1MB) has the same effect, since larger block sizes lead to higher block transmission delays and insufficient propagation. According to the measurement and analysis by Croman *et al.* [3] in 2016, given the current 10-min block interval the maximum block size should not exceed 4MB, which yields a peak throughput of 27 TPS.

2) Energy Inefficiency: As of November 2019, an average Bitcoin transaction consumes 431 KWh of electricity which can power 21 U.S. households for a day [5]. This enormous energy consumption is directly caused by the PoW-based block proposing scheme of Nakamoto consensus. As Bitcoin network's gross mining capacity grows, the Nakamoto consensus protocol has to raise the mining difficulty to maintain the average 10-min block interval, which in turn encourages miners to invest into more mining equipment with higher hashing rates. This vicious cycle shall continue as Bitcoin gains more popularity. In response, the blockchain community has come up with various block proposing schemes such as proof of stake (PoS), proof of authority (PoA), proof of elapsed time (PoET) as energy-saving alternatives to PoW.

3) Eclipse Attack: As was discussed above, the security of Bitcoin network relies on the hashing power and communication capability of honest miners. If a powerful attacker manages to dominate the in/outward communication between a victim miner and the main network (i.e., "eclipsing"), then the victim will no longer be able to contribute to the extension of the main chain [77]. Assume the percentage of hashing power controlled by the eclipse attacker, the eclipsed victims, the remaining honest miners are α, ϵ, $1-\alpha-\epsilon$, then the attacker's mining power shall be amplified to at least $\frac{\alpha}{1-\epsilon}$. If the attacker decides to exploit the eclipsed victims for growing the malicious chain, its mining power can be further enhanced up to $\alpha+\epsilon$ [78]. As a result, a double-spending attack becomes viable for the eclipse attacker if $\alpha+\epsilon > 50\%$.

Eclipse attack is in fact an exploit of the weak connectivity of permissionless peer-to-peer network based upon the Internet, which is subject to unpredictable physical bottlenecks and adversarial influences. A general approach to counter eclipse attacks is to secure the communication channels and increase the connectivity and geographical diversity of the peer-to-peer connections.

4) Selfish Mining: The 50% fault tolerance of Nakamoto consensus is built upon the assumption that all miners (both honest and malicious) strictly follow the broadcast rule that new blocks are broadcast immediately upon successful generation. If a malicious mining group withholds newly mined blocks and strategically publicizes them to disrupt the propagation of blocks mined by honest miners, they can partially nullify the work of honest miners and amplify their effective mining power. This strategy is known as *selfish mining*. It is shown by Eyal and Sirer [79] that a selfish mining group can generate a disproportionately higher revenue than that from honest mining if the group's mining power surpasses a certain threshold θ, assuming the group has a certain communication capability measured by $\gamma \in [0,1]$, which is the fraction of honest nodes that will follow the malicious chain in case of forks. As a result, the selfish mining group can attract new miners and eventually outgrow the honest miners. Notably, θ approaches zero if the selfish mining group are able to convince almost all honest miners to follow the malicious chain (i.e., $\gamma \to 1$). It is also shown that by adopting a randomized chain selection strategy at honest miners, which is equivalent to setting $\gamma = 0.5$, θ can be raised up to 25%. A later work by Sapirshtein *et al.* [80] shows that an optimized selfish mining strategy can further enhance the selfish mining pool's effective mining power fraction from α to the upper bound $\frac{\alpha}{1-\alpha}$ (achievable when $\gamma = 1$).

Selfish mining attack and eclipse attack have only happened to smaller blockchains such as Monacoin [81], but never to Bitcoin or other mainstream blockchains. This is probably due to two reasons. First, miners in established public blockchain networks actually care about the system's longevity

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

and reputation, which can positively affect the exchange rate of the cryptocurrency and thus their mining revenue. Second, established blockchains tend to be better connected (reflected by the existence of dedicated mining pools and relay networks), which allows for an effective detection of any selfish mining or eclipse attack behavior.

5) Mining Pools and Centralization Risk: According to the incentive mechanism of Nakamoto consensus, the mining revenue of a miner is proportional to its computing power. Since bitcoins can be traded for fiat currencies at exchanges, higher-earning miners have the financial advantage to purchase more efficient mining hardware, which consumes less joules per hash operation. Furthermore, higher-earning miners are often backed by large organizations that can direct huge capital into the mining business. As a result, small individual miners are either forced out of the game, or alternatively join in mining pools for stabler income. All members in a mining pool are registered with a coordinator and work to extend a common chain, while transaction validation, packaging and block proposal can be performed independently. To incentivize pool participation, block rewards are redistributed among the pool through a reliable remuneration scheme so that every pool member routinely gets a fair share of the pool's mining rewards according to its registered computing power.

In fact, joining in a mining pool has become the dominant way of participation in major PoW-based blockchains. The measurement study by Gencer *et al.* [82] in 2018 shows that throughout a one-year observation period, over 50% of the gross mining power was controlled by eight mining pools in Bitcoin and five mining pools in Ethereum. Moreover, the empirical study by Kondor *et al.* [83] in 2014 shows that the wealth distribution among Bitcoin addresses has been converging to a stable exponential distribution, and the wealth accumulation of node is positively related to its ability to attract new connections, which is another advantage of established well-connected miners.

D. Improvements to the Nakamoto Consensus Protocol

1) GHOST Rule: The greedy heaviest-observed subtree (GHOST) block finalization rule was proposed by Sompolinsky and Zohar [84] for Bitcoin in 2015. According to the longest-chain rule, all unconfirmed blocks in a fork shall be orphaned, resulting in a waste of honest mining power which could otherwise have been used to contribute to the longest-chain's security. The longest-chain rule also limits the transaction capacity since the tight tradeoff between performance and security mandates that the block interval should be sufficiently long. The GHOST rule is an alternative to the longest-chain rule that the orphaned blocks also contribute to the main chain security, effectively reducing the impacts of forks, allowing for a shorter block interval and thus higher transaction capacity. GHOST requires that given a tree of blocks with the genesis block being the root, the longest chain within the heaviest subtree shall be used as the main chain. Similar to the Nakamoto consensus, the probabilistic finality of the heaviest subtree up to the current block height will hold as long as more than 50% of mining power are honest.

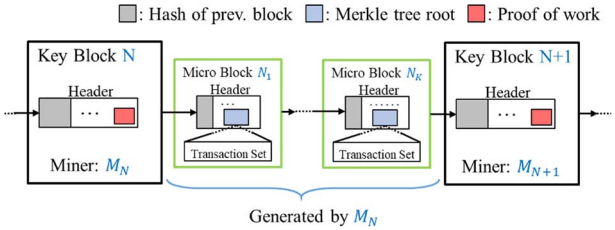


Fig. 6. Bitcoin-NG blockchain data structure. Key blocks track miners and are generated through PoW. Micro blocks curate transactions and are generated by the most recent key block miner.

The simulation result in [84] shows that given the same block interval, applying GHOST rule leads to a slightly lower transaction throughput than that with the longest-chain rule when block interval is fixed, but effectively prevents the security degradation when the block interval decreases, allowing for a higher transaction throughput. A variation of GHOST is implemented in the Ethereum blockchain, wherein the "uncle blocks" (i.e., blocks with a valid proof of work but orphaned out of the main chain) may get rewarded for their redundant mining effort. As a result, Ethereum adopts a much shorter block interval (10-15 seconds) and achieves up to 25 TPS throughput, in contrast to Bitcoin's 10-minute block interval and 7 TPS throughput.

2) Bitcoin-NG: Bitcoin-NG was proposed by Eyal *et al.* [7] in 2016 to scale up Bitcoin's transaction capacity. A variant of Bitcoin-NG called Waves-NG [85] is currently used in Wave Platform, a blockchain initiative. The key insight of Bitcoin-NG is decoupling block generation into two planes: *leader-election* and *transaction serialization*, which respectively correspond to two types of blocks: *key blocks* and *micro blocks*. The key blocks resemble Bitcoin's blocks, which contain a solution to a hash puzzle representing the proof of work and have an average block interval of 10 minutes, except for the actual transactions which are included in the micro blocks. Once a key block is mined, all subsequent micro blocks shall be generated by the current key block miner until the generation of the next key block. The generation of micro blocks is deterministic and does not contain proof of work. As a result, the micro block frequency is in the control of the key block miner (up to a maximum) to accommodate as many transactions as possible. The blockchain data structure of Bitcoin-NG is shown in Fig. 6.

The longest-chain rule is still applied to finalize and resolve forks of key blocks. As for the micro blocks, since they are batch-generated by key block miners, Bitcoin-NG relies on a combination of a heaviest-chain extension rule and a longest-chain extension rule to finalize and resolve forks of micro blocks. To encourage honest participation and discourage the current key block miner from double-spending and other malfeasance, 60% of the transaction fees collected from micro blocks by the current key block miner are redistributed to the miner of the next key block.

As for Bitcoin-NG's performance, since key blocks do not carry transactions, the transaction throughput entirely depends

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

on the micro block size and frequency. The micro block frequency needs to be controlled, for an excessive amount of them may exhaust the network bandwidth and cause frequent key block forks. Hypothetically, if we kept the key block frequency at 10 minute and micro block size at 1MB, set the micro block frequency to 12 seconds (the minimum practical block interval under current Bitcoin network condition [86]), Bitcoin-NG would achieve up to 200 TPS throughput.

On the downside, due to the determinism in micro block generation, the key block miner may become a target of denial-of-service or corruption attacks. A compromised key block miner may enclose transactions selectively or finalize contradicting transactions, the inconsistency caused by which can cost the network more than one key block cycle to remedy.

E. Hybrid PoW-BFT Consensus Protocols

The limited transaction capacity and tight tradeoff between performance and security of Nakamoto consensus are much warranted by its probabilistic finality and decentralized ideal. In contrast, BFT consensus assumes fixed participants with revealed identities and achieves deterministic finality, allowing much shorter block intervals and thus much higher transaction throughput. In response, hybrid PoW-BFT protocols have been proposed to get the best of two worlds. Here we introduce four popular proposals.

1) PeerConsensus: Proposed by Decker *et al.* [64] in 2014, PeerConsensus uses a PoW-based blockchain to throttle and certify new identities joining the network, while being agnostic to any application built upon it. The number of identities a player may control is proportional to its share of computation power, which provides Sybil resistance. With the identities established by the blockchain, the application can employ an efficient BFT protocol such as PBFT and SGMP [87] for committing transactions. The transaction fees collected are distributed to all identities equally. As a result, PeerConsensus effectively decouples participation management from transaction processing, allowing the latter to scale up throughput. On the downside, since the transaction history is not recorded in blockchain, PeerConsensus cannot control the malleability of transactions.

2) SCP: The scalable consensus protocol (SCP), proposed by Luu *et al.* [88] in 2015, incorporates BFT and sharding into blockchain consensus. The key idea of SCP is to partition the network into sub-committees (i.e., shards) with a PoW mechanism, so that each sub-committee controls a limited amount of computation power and the number of sub-committees is proportional to the network's gross computation power. This is aimed to limit the size of a sub-committee, which operates independently and curates a local blockchain using a BFT consensus protocol. A dedicated finalization committee is responsible for combining the outputs of all sub-committees into the global blockchain. A block in the global chain stores the hash and transaction Merkle tree root of every block proposed by every sub-committee. To ensure consensus safety, SCP requires each sub-committee as well as the whole network to maintain a two-thirds majority of honest computation power. However, the use of sharding and a dedicated finalization committee assumes the preexistence of network coordination, which to some extent counters the decentralized ideal of public blockchains.

Notably, using sharding to scale up blockchain transaction throughput has been extensively studied in the developer communities. Interested readers are referred to [89] for a development history and the state-of-the-arts of blockchain sharding.

3) ByzCoin: ByzCoin was proposed by Kokoris-Kogias *et al.* [90] in 2016 as a blockchain consensus protocol that leverages PoW for consensus group membership management and BFT for transaction finalization. ByzCoin takes inspiration from Bitcoin-NG's ledger structure but features a subtle difference. Instead of a linear structure, ByzCoin's ledger consists of two parallel blockchains: a keyblock chain and a microblock chain. Keyblocks are mined via PoW as in Nakamoto consensus and used for maintaining a consensus group from recent keyblock miners according to a *sliding-share-window* mechanism. Specifically, when a miner finds a new keyblock, it is credited one *share* in the consensus group and the share window moves one share forward. Only miners with shares in the window can participate in the subsequent consensus. Old shares expire once being left out of the window and so does the share owner's eligibility of consensus participation. The window length is subject to designer's choice on the consensus group size as well as the overall consensus participation fairness.

A microblock is produced by the current consensus group via an adapted PBFT protocol based on collective signing (CoSi) [91]. Compared to the original PBFT, the CoSi-based PBFT reduces the communication complexity from $O(N^2)$ to $O(N \log N)$ and thus scales better to large consensus groups. As for transaction finalization, the current keyblock miner packs up new transactions into a microblock and acts as the leader to trigger the CoSi-based PBFT. In the end, the microblock will be finalized and contain the collective signature of the consensus group and the hash pointer to the preceding keyblock, which also contains the collective signature.

ByzCoin configures that the sliding-share-window mechanism replaces one member of the consensus group at a new keyblock. This yields a slightly tighter fault-tolerance threshold than that of classical BFT consensus: $N \geq 3f + 2$ is needed anytime, where N is the consensus group size and f the Byzantine population. Meanwhile, ByzCoin's PoW-based keyblock chain is still susceptible to forks. A fork can split the consensus group and potentially make the PBFT consensus stall, which can further aggravated by the presence of selfish miners. In response, ByzCoin relies on a deterministic prioritization function tweaked with high output entropy to resolve forks timely and reduce the impact of selfish miners.

4) Pass and Shi's Hybrid Consensus: As a concurrent work to ByzCoin, the hybrid consensus protocol proposed by Pass and Shi [92] adopts a sliding-window idea similar to ByzCoin's but less susceptible to forks. That is, assuming the window size λ, the consensus group is populated by the last λ miners of the "stable part" of the blockchain, which is the main chain truncated off $\Theta(\lambda)$ blocks. This protocol keeps the PBFT

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

consensus off-chain; only the consensus epoch number and a digest of the transaction log are attached to the new block. Moreover, this protocol advocates using FruitChain [93] as the underlying PoW blockchain, which was proposed by the same authors and allegedly achieves better ledger tamper-resistance than Nakamoto's blockchain.

A Short Summary: Due to the scalability concern that BFT protocol's communication overhead would be overwhelmingly high if the consensus group grew out of control, the above hybrid PoW-BFT protocols share a common trait that the PoW mechanism is used for maintaining a stable consensus group for each BFT protocol instance. Since the PoW-based participation control does not involve actual authorization and is open to any one with computation power, we consider it a form of *soft permission* control for public blockchains. Moreover, novel signature schemes such as CoSi [91] and aggregated signature gossip [94] can help reduce communication complexity in a sparsely-connected peer-to-peer network and allow for a larger number of consensus participants.

We stress that there are more ways to hybridize PoW and BFT in addition to the above proposals. Moreover, independently proposed cryptographic techniques are often complementary to the hybrid design. This observation also applies to general hybrid PoX-BFT schemes.

VI. Proof-of-Stake Based Consensus Protocols

Proof-of-Stake (PoS) originates from the Bitcoin community as an energy efficient alternative to PoW mining. In the simplest terms, a stake refers to the coins or network tokens owned by a participant that can be invested in the blockchain consensus process. From the security point of view, PoS leverages token ownership for Sybil attack mitigation. Compared to a PoW miner whose chance to propose a block is proportional to its brute-force computation power, the chance to propose a block for a PoS miner is proportional to its stake value. From the economics point of view, PoS moves a miner's opportunity cost from outside the system (waste of computation power and electricity) to inside the system (loss of capital and investment gain) [95]. Because of the lack of real mining, we often refer to a PoS miner as a *validator*, *minter*, or a *stakeholder* for PoS's close resemblance to investing in capital markets.

We identify four classes of PoS protocols: *chain-based PoS*, *committee-based PoS*, *BFT-based PoS*, and *delegated PoS* (DPoS). Chain-based PoS inherits many of the components of the Nakamoto consensus protocol such as information propagation, block validation, and block finalization (i.e., longest-chain rule), except that the block generation mechanism is replaced with PoS. Committee-based PoS leverages a multiparty computation (MPC) scheme to determine a committee to orderly generate blocks. BFT-based PoS combines staking with BFT consensus which guarantees deterministic finality of blocks. DPoS employs a social voting mechanism that elects a fixed-size group of delegates for transaction validation and blockchain consensus on behalf of the voters. Popular examples for each PoS class are listed in Fig. 7.

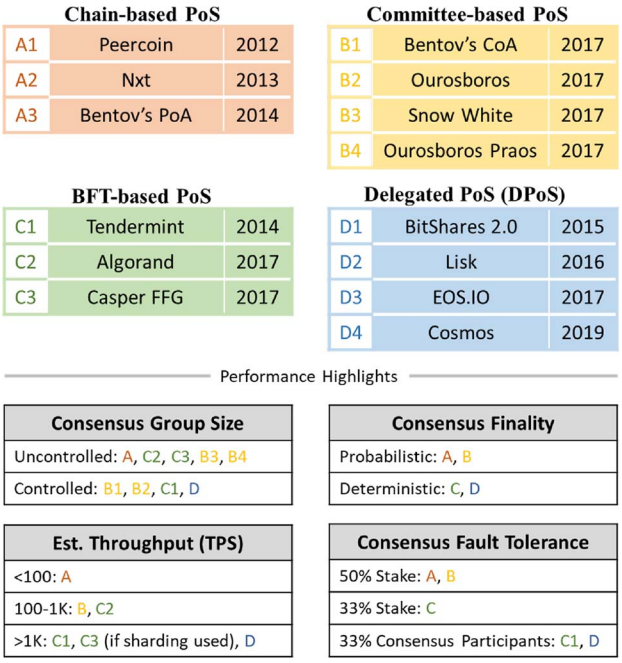


Fig. 7. Popular PoS blockchain initiatives classified under four classes and performance highlights.

A. Chain-Based PoS

Chain-based PoS is an early PoS scheme proposed by Bitcoin developers as an alternative block generation mechanism to PoW. It is within the framework of Nakamoto consensus in that the gossiping-style message passing, block validation rule, longest-chain rule, and probabilistic finality are preserved. Early full-fledged chain-based PoS blockchain systems include Peercoin and Nxt.

The general procedure of a chain-based PoS minter can be summarized by Algorithm 4. Unlike PoW, PoS does not hinge on wasteful hashing to generate blocks. A minter can solve the hashing puzzle only once for a clock tick. Since the hashing puzzle difficulty decreases with the minter's stake value, the expected number of hashing attempts for a minter to solve the puzzle can be significantly reduced if her stake value is high. Therefore, PoS avoids the brute-force hashing competition that would occur had PoW been used, thus achieving a significant reduction of energy usage.

1) Peercoin and Nxt: Both Peercoin [6] and Nxt [96] generally follow Algorithm 4. Their major difference lies in how the stake is valued. Stake value is initially proportional to stake quantity. To ensure the profitability of small stakeholders, a stake valuation scheme can be used to adjust the value of an unused stake as time passes. Peercoin uses the *coin age* metric for stake valuation, which lets the value of a stake appreciate linearly with time since the deposit. At the end of a block cycle, the value of the winner's stake returns to its base value. To avoid stakeholders from locking in a future block by deliberately waiting long, stake appreciation only continues for 90 days and stays flat since then. As a result, the chances of small stakeholders to generate a block are supplemented with time value that encourages them to stay participated even if they have not generated a block for a long time.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4: Chain-Based PoS General Procedure (Peercoin, Nxt)

Join the network by connecting to known peers;
Deposit in the stake pool;
Start *BlockGen()*;
`/* Main loop */`
while *running* **do**
  (Same with Nakamoto's protocol except that block validation should include PoS check.)
end
`/* PoS-based block generation */`
Function *BlockGen()*:
  Pack up transactions and prepare a block header context $\mathscr{C}$ containing the transaction Merkle tree root and other essential blockchain information;
  `/* PoS hashing puzzle */`
  Set up a clock (whose tick interval is a constant) and check for the following condition per clock tick:

$$Hash(\mathscr{C}|clock_time) < target \times stake_value$$

  wherein more preceding zero bits in *target* indicate a higher mining difficulty per unit of stake value;
  return new block;
end

In comparison, Nxt does not appreciate stake value continuously across block cycles like what Peercoin does. This is because the latter's coin age metric may lower the attack cost—attackers can just invest a small amount of stakes and keep attempting to generate blocks until they succeed, which is especially unfair to big stakeholders who perform honestly. Instead, Nxt requires the stake value appreciate only within one block cycle and be reset to the base value once the block cycle ends. Without coin age, Nxt addresses the monopolization problem from the incentive angle. First, stakes in Nxt are not actively managed by stakeholders; they are essentially the account balances—the more tokens held in its account, the higher the chance the stakeholder will win the right to generate a block. Second, total token supply is determined at the beginning and block rewards only come from transaction fees, which effectively aligns a stakeholder's revenue with its validation work. As a result, all stakeholders have the incentive to honestly validate transactions since it is the only way to accumulate wealth.

2) Bentov's PoA: Compared to Peercoin and Nxt, Bentov *et al.*'s proof of activity (PoA) [97] is a hybrid PoW-PoS adaptation of the Bitcoin protocol that utilizes an algorithm called *follow-the-satoshi* (FTS) to involve staking. FTS works as follows: 1) Use a pseudo-random function (PRF) to locate an atomic piece of token (e.g., satoshi in Bitcoin, wei in Ethereum) in the token universe; 2) If the atomic piece belongs to stakeholder *A*, then output *A*. With the input being a sequence of random seeds, FTS outputs a pseudo-random sequence of stakeholders such that the chance for any stakeholder to be in it is proportional to the volume of tokens owned by the stakeholder.

Bentov's PoA works as follows. At the beginning of block cycle k, an empty block header EB_k is generated according to the PoW rule and propagated across the network. After receiving EB_k, a stakeholder computes the *N*-tuple vector seed *S* as follows:

$$S_j = hash\big(hash(EB_k)|hash(B_{k-1})|SF_j\big) \quad \text{for } j = 1, \ldots, N$$

B_{k-1} is the previous block, *SF* is a *N*-tuple of fixed suffix values. *N* is a predefined value that should not be too large. Then *S* is used as the input for FTS to compute the pseudo-random sequence of stakeholders *pSeq*, which is also a *N*-tuple. Every stakeholder in *pSeq* needs to sign the block and broadcast the signature; the last stakeholder in *pSeq* wraps up the block by including transactions and the *N* signatures and broadcasts the final block B_k to the network. All stakeholders in *pSeq* shares the reward of B_k with the PoW miner of EB_k.

To avoid name conflict with proof of authority in later sections, we refer to Bentov's PoA as PoAct.

Security Analysis: Chain-based PoS can tolerate up to 50% of all stakes being maliciously controlled. And since every token can be staked, the fault tolerance further generalizes to 50% of all tokens in the network. If colluding attackers control more than 50% of stakes, they can grow their malicious chain faster than the others and carry out a double-spending attack, which is analogous to the 51% attack in PoW blockchains. However, from the economic perspective, PoS attackers have lower incentives to do so because of the capital loss risk. As staking is recorded in the form of transaction scripts, the blockchain users can retrieve the staking records according to which the consensus protocol can legally issue punishment to violators, such as nullifying stakes and disbarring the violators from participating in the future staking process.

B. Committee-Based PoS

Chain-based PoS still relies on hashing puzzles to generate blocks. As an alternative mechanism, committee-based PoS adopts a more orderly regime: determining a committee of stakeholders based on their stakes and allowing the committee to generate blocks in turns. A secure *multiparty computation* (MPC) scheme is often used to derive such a committee in the distributed network. MPC is a genre of distributed computing in which multiple parties beginning with individual inputs shall output the same result [98]. The MPC process in the committee-based PoS essentially realizes the functionality that takes in the current blockchain state which includes the stake values from all stakeholders, and outputs a pseudo-random sequence of stakeholders (we call it the *leader sequence*) which will subsequently populate the block-proposing committee. This leader sequence should be the same for all stakeholders and those with higher stake values may take up more spots in the sequence. A general procedure for a stakeholder of committee-based PoS is shown in Algorithm 5. In this algorithm, the *CommitteeElect*() functionality can also be implemented in a privacy-preserving way with verifiable random function (VRF) [99] in that only the stakeholder itself knows if it gets elected into the committee.

Well-known committee-based PoS schemes include Bentov's chain of activity (CoA), Ouroboros, Snow White,

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

Algorithm 5: Committee-Based PoS General Procedure

```
/* Joining network and staking */
Join the network by connecting to known peers;
Deposit in the stake pool;
/* Main loop */
while running do
    /* Committee election */
    if new block cycle then
        Participate in CommitteeElect();
        Check BlockGenSeq for my turns;
    end
    /* Block proposing & broadcast */
    if my turn to generate block then
        Collect transactions and generate block;
        Write block to blockchain;
        Broadcast block to the network;
    end
    /* Longest-chain&validation rule */
    if block is received & is valid & extends the longest
     chain then
        Write block into blockchain;
        Relay blocks to other committee members;
    end
end
/* PoS-based committee election */
Function CommitteeElect():
    Fetch the current blockchain state and the stake
     information of all participants; use them as the
     MPC input;
    Participate in the MPC that produces BlockGenSeq, a
     pseudo-random sequence of block generation
     opportunities;
    return BlockGenSeq;
end
```

and Ouroboros Praos. These protocols and Algorand (see Section VI-C) were developed concurrently by academics around 2017 and share many common traits.

1) Bentov's CoA: Bentov *et al.*'s CoA [100] was proposed in 2016 partially based on Bentov's PoA. It follows the main routine in Algorithm 5 in that each nominated stakeholder gets to generate its own block. CoA first leverages a MPC process to generate a string S of N random bytes. Then S is fed to a FTS algorithm that outputs a pseudo-random sequence *BlockGenSeq*. All parties should output the same *BlockGenSeq*, which is then used to coordinate the generation of the next N blocks.

2) Ouroboros: Ouroboros was developed by Kiayias *et al.* [8] in 2017 and has been used as the consensus protocol for cryptocurrency Cardano. Ouroboros divides the physical time into fixed-time epochs and each epoch is subdivided into N slots, each can be used by only one slot leader to generate a block for the network. For each epoch, stakeholders with enough stake can become electors, who will collectively elect slot leaders (i.e., the committee) for the next epoch through a MPC procedure

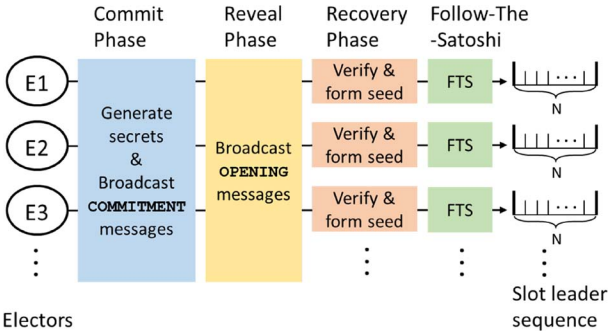


Fig. 8. PVSS-based slot leader sequence generation in Ouroboros. Secrets generated by electors at the beginning of the commit phase eventually form the random seed string. All electors shall go through FTS and obtain the same slot leader sequence at the end.

known as publicly verifiable secret sharing (PVSS), as is shown in Fig. 8. In the commit phase, elector E_i broadcasts a commitment message that includes a random secret. In the reveal phase, E_i broadcasts an opening message that reveals the previously sent secret. In the recovery phase every elector verifies that commitments and openings match and then form a seed string with the revealed secrets. All electors have the same seed string and shall obtain the same slot leader sequence after executing FTS. As we shall see next, Ouroboros' one-slot-one-leader arrangement entails stringent network synchrony, and the PVSS-based leader selection may expose the elected leaders to targeted attacks.

3) Ouroboro Praos: Ouroboros Praos was proposed by David *et al.* [101] in 2017 to address two security concerns of Ouroboros. First, Ouroboros requires stringent network synchrony for slot leaders to use their allocated slots precisely, which is vulnerable to desynchronization attacks. In comparison, Ouroboros Praos is designed for partially synchronous networks wherein a maximum delay of Δ slots is allowed for message delivery, albeit Δ is unknown to electors. This is achieved by allocating empty slots that help electors to re-synchronize to the network and allowing a slot to have multiple slot leaders. Second, Ouroboros is susceptible to adaptive corruption against slot leaders. Since the leader sequence for the next epoch is known to all network participants, an attacker may target and try to corrupt slot leaders ahead of their block proposing. In comparison, Ouroboros Praos employs a locally executed verifiable random function (VRF) [99] that allows only the elector herself to know her block proposing slots for the next epoch, which are verified by the corresponding VRF proofs. Compared to Ouroboros' PVSS-based leader selection, the VRF scheme saves much of the communication overhead at the cost of local cryptographic computation. Similar schemes are also used in contemporary protocols such as Snow White and Algorand.

Because of its reduced synchrony requirement and the privacy-preserving nature of the VRF scheme, Ouroboros Praos does not limit the size of consensus participants and allows for a flexible committee. Ouroboros Praos also adopts key-evolving signatures (KES) to counter posterior corruption and provide forward security, which we will elaborate in Section VI-E.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

4) Snow White: Snow White was developed by Daian *et al.* [9] in 2017. It is a PoS protocol specifically designed to accommodate the sporadic participation model in which nodes can switch online/offline arbitrarily. Similar to the committee-based PoS schemes above, Snow White employs a MPC procedure to decide the block proposing committee, each of which is issued an eligibility ticket privately. For each epoch, every stakeholder takes the current blockchain (including staking information) as input and output the committee for the next epoch. Specially, Snow White executes a modified version of the sleepy consensus protocol [102], an asynchronous consensus protocol that ensures the consensus safety in case of sporadic participation and committee reconfiguration. Compared to contemporary schemes such as Ouroboros, Ouroboros Praos and Algorand, this feature enables Snow White to work under harsh network conditions with frequent disconnections and volatile message delays. Moreover, Snow White uses a checkpointing scheme to finalize earlier history that protects the blockchain from posterior corruption attack [97] and adaptive key selection attack.

Security Analysis: In spite of having an orderly block proposing scheme, committee-based PoS still adheres to the longest chain rule for probabilistic finality. So long as fewer than 50% stakes are held by the malicious party, the honest parties can safely maintain the longest chain. Meanwhile, the expansion of the committee may lead to deterioration in network connectivity which can result in a significant drop in protocol performance and desynchronization of block proposal. The round-based committee election process with a predetermined round duration (e.g., Ouroboros' fixed timeslot) also faces scalability problems, as large committee sizes may lead to never ending consensus cycles due to the excessive communication overhead. To mitigate such risks, the duration of a communication round can be extended sufficiently to ensure that all broadcast messages are delivered before the participants proceed to the next round. This however leads to longer transaction conformation latency and lower throughput. A more straightforward approach is limiting the committee size by imposing a minimum stake requirement for the committee members. For example, Cardano, the cryptocurrency platform that deploys Ouroboros, mandates that every committee member should own no less than 2% of total tokens in circulation. This effectively limits the committee size to 50, safeguarding an efficient consensus process.

C. BFT-Based PoS

Chain-based PoS and committee-based PoS largely follow the Nakamoto consensus framework in that the longest-chain rule is still used to provide probabilistic finality of blocks. In comparison, BFT-based PoS (or hybrid PoS-BFT) incorporates an extra layer of BFT consensus that provides fast and deterministic block finalization. Algorithm 6 shows the general procedure of BFT-based PoS at every participant. Block proposing can be done by any PoS mechanism (round-robin, committee-based, etc.) as long as it injects a stable flow of new blocks into the BFT consensus layer.

Algorithm 6: BFT-Based PoS General Procedure

```
Join the network by connecting to known peers;
Start BlockGen();
/* Main loop                                   */
while running do
    /* Block proposing & broadcast             */
    if BlockGen() returns block then
        Add block to its tempBlockSet;
        Broadcast block to the network;
    end
    /* Block validation                        */
    if block is received & is valid then
        Add block to its tempBlockSet;
        Relay block to the network;
    end
    /* BFT consensus layer                     */
    if new consensus epoch then
        Perform BlockFinBFT() on tempBlockSet;
        Write the winning block to blockchain;
        Clear tempBlockSet;
    end
end
/* PoS-based block generation                  */
Function BlockGen():
    Elect a block proposer, whose success rate is
     proportional to stake value;
    Propose block;
    return block;
end
/* BFT-based block finalization                */
Function BlockFinBFT():
    Participate in a BFT consensus that finalizes one
     winning block out of tempBlockSet;
    return the winning block;
end
```

Aside from the general procedure, a checkpointing mechanism can be used to seal the finality of the blockchain (not shown in Algorithm 6). As a result, the longest-chain rule can be safely replaced by the most-recent-stable-checkpoint rule for determining the stable main chain. Popular BFT-based PoS blockchain protocols include Tendermint, Algorand, and Casper FFG. DPoS protocols such as EOSIO also use BFT consensus for block finalization within delegates.

1) Tendermint: Tendermint was developed by Kwon [12] and Buchman [103] in 2014 and currently used in Cosmos Hub network [104]. It is the first public blockchain project to incorporate a BFT consensus layer and takes inspiration from the DLS protocol [28] and PBFT [27]. Tendermint works in consensus cycles. Each cycle involves a multi-round BFT consensus process to finalize one block. Each round consists of three phases: Propose, Prevote, Precommit. Specially, in the Propose phase a validator is designated by a deterministic algorithm as the block proposer in a round-robin fashion such that validators are chosen with frequency proportional to the value of their deposited stakes. A validator

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

continues iterating the three-phase rounds until one block receives more than 2/3 of Precommits. The validator then broadcasts Commit votes for the block and listens for other validators' Commit votes. When a block receives more than 2/3 of Commit votes, it will be finalized in blockchain. As long as more than 2/3 of validators of each round are honest, Tendermint can achieve consensus safety. On the other hand, because Tendermint selects block proposers deterministically, the future block proposers are susceptible to targeted attacks (e.g., DDoS). Tendermint addresses this risk by deploying sentry nodes which act as proxies of block proposers and never reveal the IP addresses of the latter [105]. Notably, since Tendermint decouples the PoS mechanism from the BFT layer, a validator's stake value does not add weight to its consensus votes. For this reason, Tendermint is often considered an early effort on applying BFT consensus to blockchain.

Tendermint also features an equal-sharing-style incentive mechanism instead of winner-takes-all. For every block height, the block reward is distributed among the block proposer and validators from whom the proposer received Commit votes. However, fairness may be impaired if Commit votes are not delivered in time before the next cycle, as is demonstrated by Amoussou-Guenou *et al.* [106].

2) Algorand: Algorand is a cryptocurrency system developed by Gilad *et al.* [13] at MIT CSAIL in 2017. It employs committee-based PoS for block proposing and Byzantine agreement for block finalization. First, similar to Ouroboros Praos' block proposer election mechanism, the election of Algorand's block proposing committee is done by a VRF scheme called *cryptographic sortition* which sorts candidates according to the amount of coins they own. Only those with rankings above a threshold are admitted into the committee for the next block cycle. Every individual user can check privately if it is in the committee. At each user *i*, cryptographic sortition outputs an eligibility proof signed by the user's private key $\sigma_{sk_i}^{ep}$ showing that it is truly a committee member. $\sigma_{sk_i}^{ep}$ is broadcast to the network along with the new block proposed by user *i*. Upon receiving the block, other users can verify the proof via user *i*'s public key pk_i.

On top of the cryptographic sortition-based block proposing scheme, Algorand relies on a Byzantine agreement protocol called *BA*$\star$ for block finalization. *BA*$\star$ reduces the consensus problem to binary Byzantine agreement: either agreeing on a proposed block or an empty block. In the ideal case where strong network synchrony is assumed, the committee follows *BA*$\star$ to exchange votes on proposed blocks so that they will decide a final block, or an empty block if no blocks pass the eligibility proof check. In a weakly synchronous network where block propagation and message exchange among committee members can suffer from uncertain delays, *BA*$\star$ outputs tentative blocks if none of the proposed blocks can be finalized, which results in a fork. To resolve the forks of tentative blocks, Algorand periodically runs a recovery protocol to accept a tentative block if there is any. Specially, the recovery protocol needs to be invoked by a synchronized committee. Therefore, according to the authors [13], weak synchronony is sufficient for consensus safety during *BA*$\star$'s routine operation while strong synchrony is required for consensus liveness when kicking off the recovery protocol for resolving forks.

Algorithm 7: Casper FFG

```
Deposit in the stake pool;
/* Main loop                                          */
while running do
    (Block proposing and block validation are the same
     as in Algorithm 6, except that blocks are attached to
     BlockTree rather than stored in a temporary set.)
    /* BFT consensus layer                            */
    if new consensus epoch then
        Identify valid checkpoint blocks and attach them
         to CheckPointTree;
        Participate in CheckPointVote() w.r.t.
         CheckPointTree, which returns CP_s, CP_t;
        Mark CP_s finalized and CP_t justified;
    end
end
/* Staked checkpoint voting                           */
Function CheckpointVote():
    Broadcast a vote for a source-target checkpoint pair
     in CheckPointTree;
    Check received votes against the slashing rules and
     then evaluate them by signer's deposited stake;
    if pair ⟨CP_s, CP_t⟩'s votes cover more than 2/3 of
     total deposited stakes then
        return CP_s, CP_t;
    end
end
```

3) Casper FFG: Casper FFG was first envisioned by Zamfir [107] in 2015 and formally presented by Buterin and Griffith [14] in 2017. It is a light-weight PoS consensus layer built on top of Ethereum's current PoW-based block proposing mechanism (Ethash). Casper FFG slightly deviates from Algorithm 6 for that it directly incorporates PoS into block finalization. Algorithm 7 shows the general procedure of Casper FFG for each validator. Newly generated and received blocks are attached to the *BlockTree*, which is similar to the tree data structure used by the GHOST rule. The actual consensus subject, however, is the *CheckPointTree*, which is a subtree of *BlockTree*. Specifically, for every consensus epoch (100 in *BlockTree*'s height or 1 in *CheckPointTree*'s height), every validator broadcasts to peers a vote for a block as the checkpoint. The height of the block in *BlockTree* must be divisible by 100. The vote consists of a *justified* source checkpoint CP_s and its height $h(s)$, a target checkpoint CP_t and its height $h(t)$ ($h(s) < h(t)$), and the validator's signature *S*. All votes are broadcast to the network and are weighted by the signer's stake value. If the source-target checkpoint pair $\langle CP_s, CP_t \rangle$ is voted by validators who possess more than 2/3 of total deposited stakes, then CP_t is justified and CP_s is *finalized*. All blocks between CP_s and CP_t are finalized as well. Casper FFG relies on two so called *Casper Commandments* for ensuring consensus safety: 1) validator must not cast two distinct votes for the same checkpoint height, and 2) validator must not cast a

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

new vote whose source-target span is within that of its existing vote. Violators are subject to slashing rules including forfeiting stakes and temporarily banning from staking. Since every vote is signed with the validator's private key and received by peer validators, Casper FFG can conveniently detect violators and enforce the slashing rules.

The current smart contract implementation of Casper FFG is documented in EIP 1011 [70]. A stakeholder becomes a participating validator by depositing a stake in the dedicated smart contract, which encodes Casper FFG and can be accessed via Ethereum transactions. Notably, Casper FFG is the preamble project of Casper Correct-by-Construction (Casper CBC), the PoS protocol family that will be used by Ethereum 2.0 to complete the transition to pure PoS [108].

To further improve performance and scalability, Ethereum 2.0 also plans to combine PoS with sharding [109]. In a nutshell, all Ethereum 2.0 participants are divided into shards. Each shard runs a blockchain instance via a consensus scheme not limiting to PoS. On the top level, the main chain, known as the "beacon chain", will be maintained by a group of known validators via a Casper CBC protocol. Each validator is randomly assigned to a shard as the shard manager and periodically commits a digest of the shard chain to the main chain. The parallelism of sharding and the energy efficiency of PoS can theoretically scale up both transaction throughput and network size. Nonetheless, the sharding scheme is still an ongoing work and faces several challenges before being harmonized with Casper CBC. They include the increased take-over risk related to small shard size, the difficulty in coordinating inter-shard communication and token transfer.

Security Analysis: BFT-based PoS's consensus fault tolerance varies among the three above-mentioned implementations. In Tendermint, although block proposers are determined based on PoS, all validators have the equal weight in the consensus process. Therefore Tendermint tolerates up to 1/3 of Byzantine validators. In comparison, Algorand and Casper FFG tolerate up to 1/3 of maliciously-possessed stakes. In Algorand, if an attacker owns more than 1/3 of total tokens, then chance is high that more than 1/3 of the elected committee members are compromised by the attacker, leading to consensus failure of $BA\star$. In Casper FFG, if an attacker owns more than 1/3 of total deposited stakes and dominates the communication within the network, the compromised validators can vote on conflicting checkpoints without getting punished. Since a typical BFT consensus protocol can incorporate a checkpointing mechanism to ensure deterministic finality of blocks, costless simulation attacks can be naturally avoided (to introduce in Section VI-E).

D. Delegated PoS (DPoS)

DPoS can be seen as a democratic form of committee-based PoS in that the committee (consensus group) is chosen via public stake delegation. It is currently used by BitShares 2.0 (2015) [110], Lisk (2016) [111], EOSIO (2017) [10], and Cosmos [104]. DPoS was designed to control the size of the consensus group so that the messaging overhead of the consensus protocol remains manageable. Members of the consensus group are also called *delegates*. The election of delegates is called the *delegation* process, and a general example is shown in Fig. 9. In the actual case, the delegation process and the soliciting of votes may involve outside incentives. And the delegation process may turn out to be an interesting socioeconomic phenomenon.

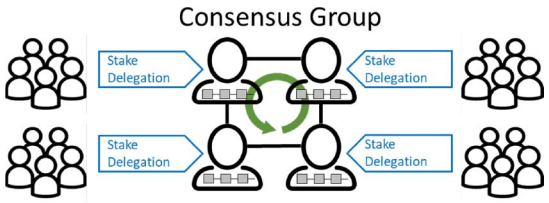


Fig. 9. Illustration of the delegation process in DPoS. Small stakeholders vote for delegates with their stakes and the most voted delegates form the consensus group.

Generally, an aspiring delegate needs to attract enough votes from normal token holders. This is often accomplished by offering a popular application and building up reputation through propaganda campaigns. By casting a vote to a delegate via a blockchain transaction, a token holder entrusts the delegate with its own stake. As a result, the delegate harvests the stake voting power from her voters and acts as their proxy in the consensus process. A token holder can switch vote to another delegate via a new delegation transaction. Take EOSIO for example, the EOSIO protocol mandates that anyone can be a delegate and solicit votes, but only those who ascend to top 21 join the consensus group, among whom the right of block proposal is equally shared. EOSIO employs a pipelined PBFT-style consensus scheme to finalize the proposed blocks across the 21 delegates [112]. Specially, the physical time is divided into slots and the 21 delegates take turns to propose a block in a round-robin fashion. At each slot when a delegate proposes a block, the consensus scheme goes through pre-commitment and commitment phases and decides on the fate of the proposed block. Because of the small consensus group size and orderly PBFT-style procedure, every new valid transaction can be near-instantly propagated to the consensus group and finalized in the blockchain.

To enforce transaction validation and consensus safety, DPoS's incentive mechanism is designed to encourage honest delegation and consensus participation. Every delegate receives a daily vote-reward proportional to the votes she has. Once ascending to the consensus group, delegates also receive block rewards for validation work.

Security Analysis: Assuming BFT is used by the consensus group for block finalization, which is recommended since the group size is limited, DPoS can tolerate 1/3 of delegates being malicious. For example, EOSIO can tolerate at most 6 out of 21 delegates being malicious. In the real world they may not wish to misbehave or collude at all, since all delegates have revealed their identities to voters and would be scrutinized for any misconduct.

E. Vulnerabilities of PoS

Although heralded as the most promising mechanism to replace PoW, PoS still faces several vulnerabilities.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

1) Costless Simulation: Costless simulation is a major vulnerability of non-BFT-based PoS schemes, especially chain-based PoS in which PoS is used to simulate the would-be PoW process. Costless simulation literally means any player can simulate any segment of blockchain history at the cost of no real work but speculation, as PoS does not incur intensive computation while the blockchain records all staking history. This may give attackers shortcuts to fabricate an alternative blockchain. The four subsequent vulnerabilities, namely *nothing-at-stake*, *posterior corruption attack*, *long-range attack*, and *stake-grinding attack* are all based on costless simulation.

2) Nothing-at-Stake: Nothing-at-stake is the first identified costless simulation problem that affects chain-based PoS. It is also known as "multi-bet" or "rational forking" problem. Unlike a PoW miner, a PoS minter needs little extra effort to validate transactions and generate blocks on multiple competing chains simultaneously. This "multi-bet" strategy makes economical sense to PoS minters because by doing so they can avoid the opportunity cost of sticking to any single chain. Consequently if a significant fraction of nodes perform the "multi-bet" strategy, an attacker holding far less than 50% of tokens can mount a successful double spending attack [113]. Nothing-at-stake problem can be practically solved by penalizing whoever multi-bets, such as forfeiting part of or all their stakes. However the penalties could still be reversed if the attacker eventually succeeds in growing a malicious chain.

3) Posterior Corruption: Dubbed by Bentov *et al.* [97] as "bribing attack" in 2014, posterior corruption is another attack utilizing costless simulation against PoS. The key enabler of posterior corruption is the public availability of staking history on the blockchain, which includes stakeholder addresses and staking amounts. An attacker can attempt to corrupt the stakeholders who once possessed substantial stakes but little at present by promising them rewards after growing an alternative chain with altered transaction history (we call it a "malicious chain"). When there are enough stakeholders corrupted, the colluding group (attacker and corrupted once-rich stakeholders) could own a significant portion of tokens (possibly more than 50%) at some point in history, from which they are able to grow an malicious chain that will eventually surpass the current main chain. Since posterior corruption is only possible because the private/public keys are fixed for blockchain participants, key-evolving cryptography (KEC) [114] can be applied so that the past signatures cannot be forged by the future private keys. Ouroboros Praos [101] currently adopts KEC for this purpose. Alternatively, as is in Snow White [9] and Casper FFG [14], checkpointing can be used to finalize the ledger and eliminate the possibility of posterior corruption.

4) Long-Range Attack: Coined by the Ethereum founder Buterin [115], long-range attack can be viewed as the ultimate form of costless simulation. It foresees that a small group of colluding attackers can regrow a longer valid chain that starts not long after the genesis block. Because there were likely only a few stakeholders and a lack of competition at the nascent stage of the blockchain, the attackers can grow the malicious chain very fast and redo all the PoS blocks (i.e., by costless simulation) while claiming all the historical block rewards. If the blockchain network is not incentivized by block rewards, the attackers can still deploy a similar long-range scheme called *stake-bleeding attack* [116] as long as transaction fees exist. That is, the attackers can accumulate significant amount of wealth by collecting most of historical transaction fees through the malicious chain. Through either scheme, once the malicious chain overtakes the main chain, it is released to public and becomes the new main chain.

While zero transaction fees can counter stake-bleeding attacks, general long-range attacks (and costless simulation as a whole) can be resolved by checkpointing, a more radical measure. Checkpointing is widely used in BFT protocols to ensure the finality of system agreements and safely discard older records. For permissionless blockchains (the main venue for chain-based PoS), however, checkpointing can undermine decentralization, as the finality of checkpoints always requires the endorsement from certain authoritative entities.

5) Stake-Grinding Attack: Generally, the block generation competition in Nakamoto-style blockchain with proof-of-X block proposal is a pseudo-random process that a higher X (e.g., work, stake) yields a higher probability of winning the competition. However, unlike PoW in which pseudo-randomness is guaranteed by the brute-force use of a cryptographic hash function, PoS's pseudo-randomness is influenced by extra blockchain information—the staking history. Malicious PoS minters may take advantage of costless simulation and other staking-related mechanisms to bias the randomness of PoS in their own favor, thus achieving higher winning probabilities compared to their stake amounts [8]. For example, in chain-based PoS blockchains such Peercoin, at a certain historical point of the blockchain, attackers may iterate through different block headers and increase the probability of generating a valid block with their stakes [117]. For committee-based and BFT-based PoS schemes that decouple the election of block proposers from block generation, stake-grinding attacks can be mitigated by ensuring the PoS pseudo-randomness with a secure block proposer election scheme that involves minimal local information. Examples include Ouroboros, Ouroboros Praos, and Algorand.

6) Centralization Risk: PoS faces a similar wealth centralization risk as with PoW. In PoS the minters can lawfully reinvest their profits into staking perpetually, which allows the one with a large sum of unused tokens become wealthier and eventually reach a monopoly status. When a player owns more than 50% of tokens in circulation, the consensus process will be dominated by this player and the system integrity will not be guaranteed. Take Ethereum's Casper FFG for example, the proposed PoS scheme is built upon the current PoW system, of which the cryptocurrency ethers can be directly used for staking. This gives initial advantages to those who have already accumulated huge wealth during Ethereum's PoW operation. Potential countermeasures against monopolization in PoS mainly come from the incentive mechanism and economic perspective. In addition to the stake valuation scheme that improves the winning chances of small stakeholders (see Section VI-A), we can use off-chain factors to complicate the staking process (EOSIO for example) and

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

impose taxation on the blocks generated by large stakeholders, to name a few.

VII. Other Emerging Blockchain Consensus Mechanisms and Protocols

The majority of contemporary blockchain consensus protocols make use of established schemes including PoW, PoS, and BFT. There are numerous other promising proposals for specific application scenarios, many of which are still conceptual and cover just one or two of the five blockchain consensus protocol components. In this section we present five of such proposals, namely proof of authority (PoA), proof of elapsed time (PoET), proof of TEE-stake (PoTS), proof of retrievability (PoR), Ripple consensus protocol/algorithm (RCPA). A comprehensive review of two types of DAG-based protocols, namely blockDAG and txDAG, are also provided.

A. Proof of Authority (PoA)

Proof of authority was coined by Ethereum co-founder Gavin Wood as an alternative to PoW and PoS. It is currently deployed in Ethereum's Rinkeby (2017) and Kovan (2017) testnet, and POA Network (2018) [11]. In a nutshell, PoA is a special case of PoS in that a validator stakes with its identity instead of monetary tokens. To qualify as a PoA validator in the consensus group, a participant needs to go through a mandatory certification process to build up its authority. It generally involves having the unique identity verified, demonstrating the ability to contribute consistently to the consensus, and making all certification documents publicly available. The consensus group should be stable and small in size, and publicly scrutinized so that users can entrust the consensus group for reliable transaction processing and blockchain curation. If a validator shows incompetence in such tasks or misbehaves, it will be discredited by users and peer validators.

Security Analysis: The fault tolerance of a PoA blockchain depends on the consensus protocol used by the consensus group. Besides BFT protocols with 1/3 fault tolerance threshold, Nakamoto-style protocols such as Parity's AuthorityRound (AuRa) [118] can also be used to tolerate up to 50% of colluding validators. Featured by its small but trusted consensus group, PoA is a good example of trading decentralization for security and performance. To prevent validators from collusion, they are required to operate independently and constantly monitored by users.

B. Proof of Elapsed Time (PoET)

PoET was proposed by Intel as an alternative mining mechanism in 2016 and subsequently used in the Hyperledger Sawtooth family [119]. Instead of undergoing the hashing-intensive mining, PoET simulates the time that would be consumed by PoW mining. That is, every node randomly backs off for an exponentially distributed period of time before announcing its block. To ensure that the local time truly elapses, PoET requires the back-off mechanism to be carried out in a trusted execution environment (TEE), which is an isolated memory area that provides integrity and confidentiality to the program running inside, against a compromised hosting platform. Specially, the program enclosed in a TEE is called an "enclave". Intel SGX [120] and Arm TrustZone [121] are the two major TEE solutions. Among other utilities, TEE provides an integrity proof of enclave program through remote attestation, which essentially helps the network establish trust on consensus participants.

Taking Hyperledger Sawtooth for example, the PoET protocol works in two phases for every participating node *i*:

1) *TEE setup:* Node i first obtains the PoET protocol program from a trusted source and instantiates the program on its SGX machine wherein the random back-off routine runs inside an enclave. The trusted program generates a signing key pair $\langle PK_i, SK_i \rangle$ for node i and starts the attestation process which results in sending an attestation report to the network. The attestation report contains the public key PK_i and the enclave measurement signed by the Intel Enhanced Privacy Identification (EPID) private key inside the enclave. Other nodes in the network will validate the node *i*'s hardware authenticity through Intel Attestation Service (IAS) and validate the attestation report before accepting node *i*.
2) *Participating in consensus:* The consensus process is similar to Nakamoto except for block generation and validation. For each block cycle, node *i* waits for a period dictated by the random back-off routine running in the enclave before producing a new block. The enclave then generates a certificate of back-off completion signed by node *i*'s private key SK_i which is broadcast to the network along with the new block. Upon receipt of the new block, other nodes validate the block content and the back-off completion certificate with node *i*'s public key PK_i. A block shall be appended to the blockchain if it passes validation and the finalization rule.

While PoET was initially proposed to substitute the PoW for block proposal without touching on the longest-chain rule, the usage of hardware-assisted public key cryptography actually enables the use of more efficient BFT algorithms for block finalization. A hybrid PoET-BFT scheme is currently used by Sawtooth PBFT [122], a sub-project of Hyperledger Sawtooth.

Security Analysis: PoET can tolerate up to 50% TEE nodes being malicious. Since every TEE node runs the same random back-off routine in its enclave, a player can shorten its expected back-off time by running multiple TEE nodes, which is susceptible to Sybil attacks. If more than 50% of TEE devices collude or are controlled by an attacker, they can ultimately win the block race and keep extending the malicious chain. Therefore, PoET is most suited for permissioned blockchains, where every participant is authenticated and runs one TEE node.

For Hyperledger Sawtooth, Intel is the sole TEE hardware vendor and attestation service provider, which poses a single point of risk to the network. The validity of remote attestation depends on the integrity of SGX implementation and the availability of IAS. Recent attacks such as *cache*

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

attacks [123], *Foreshadow* [124] and *Foreshadow-NG* [125] have demonstrated the ability to extract the EPID private key from hardware exploiting side channels and speculative execution, posing a security threat at the hardware level.

C. Proof of TEE-Stake (PoTS)

PoTS is another protocol harmonizing TEE and blockchain consensus. It was first proposed by Li *et al.* [126] in 2016 and formalized by Andreina *et al.* [127] in 2019. A PoTS node *i* follows the same setup procedure as in PoET to bootstrap a TEE enclave, generate the signing key pair $\langle SK_i, PK_i \rangle$, and attest the setup to the network. Instead of simulating the would-be elapsed time of PoW mining, the enclave program of PoTS is akin to Algorand's cryptographic sortition scheme that randomly selects a committee according to the stake distribution. Every node in the committee is eligible to propose a new block for the coming block cycle. To prove the block proposal eligibility to the network, the block includes the eligibility proof signature $\sigma^{ep}_{SK_i}$, which is produced by the enclave program of the block generating node *i*. Once other nodes receive this block, they validate the block content as well as signature $\sigma^{ep}_{SK_i}$ using node *i*'s public key PK_i. The longest-chain rule is then used to determine whether to accept this block into the blockchain.

Security Analysis: PoTS can tolerate up to 50% of all stake value at TEE nodes being maliciously controlled, the same as the fault tolerance of chain-based or committee based PoS. Compared to PoET, the incorporation of staking gives PoTS higher robustness against Sybil attacks, which implies its applicability to permissionless blockchains. Furthermore, the security offered by public key cryptography and TEE-certified execution of committee selection helps PoTS counter the stake-bleeding and stake-grinding attack. The single point of risk in TEE hardware vendor still exists.

D. Proof of Retrievability (PoR)

PoR was originally proposed by Juels and Kaliski, Jr., [128] in 2007 as a cryptographic building block for a semi-trusted distributed archiving system. The core feature of PoR is to allow a file owner to check if its online files or file fragments are securely stored and retrievable through a challenge-response protocol. The retrievability of a target file *F* at a remote node n_i can prove n_i indeed spends the required amount of storage resources on *F*. Because of the space requirement behind retrievability, PoR is also known as proof of space.

In the role of a consensus protocol, PoR was first used by the cryptocurrency Permacoin, proposed by Miller *et al.* [129] in 2014. It was designed as a mining-free alternative to PoW. First, a central dealer publishes a target dataset *F* and computes the digest of *F* (the Merkle hash tree root of all segments of *F*). Then each participant stores some random segments of *F* per its storage capability, and computes the digest of these segments. For every block cycle, the dealer initiates a lottery game with a random puzzle. Then every participant derives a lottery ticket consisting of a fixed number of PoR challenges from its locally stored segments, public key, and the puzzle. Participants with more segments stored have higher probability of winning the lottery and thus being eligible to generate a block. All PoR challenges are stored in the new block and verified by the whole network. Permacoin also implements a signature scheme to discourage participants from outsourcing the storage task. Aside from PoR, Permacoin inherits Bitcoin for other consensus components.

Compared to PoW, PoR has two economical advantages. First, file storage consumes far less energy than brute-force mining, and storage space as a resource can be recycled. Second, PoR can be repurposed for meaningful storage tasks. For example the target dataset can be some extremely large but useful public dataset. In fact, the latter advantage is not seen in any other proof-of-X schemes.

Security Analysis: Since the block winning rate of a participant is proportional to its local storage space, PoR can tolerate up to 50% of gross storage being held up by the malicious party. Although this still can trigger an arm race of storage resources, it downplays the efficacy of ASICs and encourages a wider variety of mining participants. Meanwhile, the 50% threshold depends on the randomness of the central dealer's job. To ensure the diversity of lottery tickets across all participants and increase the randomness of the lottery, the target dataset should be large enough so that participants stores almost non-overlapping segments. This assumption can be undermined if the dealer chooses a dataset not large enough or deliberately distributes overlapping segments.

E. Ripple Consensus Protocol/Algorithm (RCPA)

RCPA was proposed by Schwartz *et al.* [130] in 2014 as the underlying protocol for Ripple, a global payment and gross settlement network operated by the Ripple company. Unlike public blockchains such as Bitcoin and Ethereum, Ripple treats individual transactions as the ledger's atomic items, similar to a transaction log.

In Ripple network, only validator nodes can participate in consensus. We call a validator node "node" for simplicity. Nodes collect transactions from clients and propose them to peer nodes for consensus. In initialization, every node establishes a unique node list (UNL) which identifies the nodes it can trust and directly exchange messages with. A UNL relationship is reciprocal. We call a group of nodes that are fully connected by UNL relationships a UNL clique.

The operation of RCPA at one node is shown in Algorithm 8. RCPA runs in epochs, each of which finalizes a certain set of transactions into the ledger. Every epoch involves multiple rounds of transaction filtering. The ledger is marked "closed" after the final round. Specially, the yes-vote threshold of the final round $TH_{MaxRound} = 80\%$, which is designed to cope with the network connectivity assumption that any two UNL cliques UNL_i, UNL_j must be at least 20% overlapped, i.e., sharing at least $\frac{1}{5}\max(|UNL_i|, |UNL_j|)$ inter-clique UNL relationships. In an ideal environment, as long as *MaxRound* is large enough, all valid transactions proposed by non-faulty nodes should eventually surpass the 80% yes-vote threshold.

Security Analysis: RCPA imitates a relaxed DLS protocol with an artificial BFT bound of 1/5. It further requires no more

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

Algorithm 8: RPCA (validator Node)

```
Joining Ripple network as a validator;
/* Main loop                                          */
for new epoch do
    Collect valid transactions (new or leftover from
     previous epochs) ⇒ CandidateSet;
    for r = 1 → MaxRound do
        Broadcast CandidateSet to UNL peers;
        After receiving transactions from UNL peers, add
         them to CandidateSet and broadcast a vote on
         the veracity (yes/no) of every transaction;
        After receiving votes from UNL peers, discard
         transactions from CandidateSet whose
         yes-votes fall short of a threshold TH_r
         (TH_MaxRound = 80%);
    end
    The remaining transactions in CandidateSet are
     accepted into the ledger;
end
```

than 1/5 of nodes are faulty in every UNL clique in order to ensure overall network consensus. Compared to PBFT that achieves 1/3 Byzantine fault tolerance with $O(N^2)$ message complexity in a fully connected network, RPCA's 1/5 fault tolerance bound trades for a lower connectivity requirement and thus lower message complexity per block cycle, which is $O(MK^2) = O(NK)$ where *K* is the clique size and $M = N/K$ is the number of cliques. Therefore Ripple demonstrates the possibility of trading fault tolerance for better performance when a certain level of trust is assumed. On the down side, RCPA's multi-round broadcast scheme among a UNL clique and the quick convergence of votes require high synchrony among clique members. This impairs Ripple's decentralization capability in practical settings.

Ripple's current customers are primarily established corporations and financial institutions. Possible reasons include the above-mentioned synchrony requirement and that the 1/5 fault tolerance can be too restrictive for low-trust environments. Interestingly, Stellar [131], originally a fork project of the Ripple, has shifted to federated Byzantine quorum systems (FBQS) for a loose synchrony requirement on participants and achieves the 1/3 BFT bound [132].

F. BlockDAG-Based Consensus Protocols

Starting from 2016 there have been increasing interests in non-linear ledger structures for the aim of better performance, among which directed acyclic graph (DAG) has received the most attention. Consensus schemes with DAG ledger structure mark a significant divergence from Nakamoto's blockchain design. Their key insight is that transaction throughput should not be limited by a restrictive consensus object, such as a linearly growing chain of blocks with fixed time intervals. Instead, the influx of transactions should drive the ledger expansion. There are two types of DAG ledgers: block-based DAG (blockDAG) and transaction-based DAG (txDAG). In this subsection we focus on blockDAG and defer the latter type to the next subsection.

In a blockDAG, every vertex contains a collection of transactions which is similar to the block concept in blockchain. What sets blockDAG apart from blockchain is that every block can be hash-pointed to multiple parent blocks. This leads to a situation that every new block can be appended to the DAG with considerable flexibility on how many and which parents to point to. This parent-selecting conundrum is commonly regarded as the major challenge for DAG-based consensus schemes and pertains to the system's transaction processing capability and security against Sybil and double-spending attacks. Next we introduce two concurrently proposed blockDAG schemes, SPECTRE and PHANTOM, with a focus on the parent-selection mechanism.

1) SPECTRE: Proposed by Sompolinsky *et al.* [133] in 2016, SPECTRE is one of the first well-documented blockDAG proposals. SPECTRE requires any node who wants to mine a new block to find all blocks of zero in-degree (i.e., "tips") in the DAG and hash-point the new block header to these tips before starting the PoW mining for the new block. The node broadcasts the newly mined block to the network. Every node initiates a recursive voting procedure to determine the order of any two blocks in the current DAG. This recursive procedure eventually results in a pairwise ordering over the blockDAG. Every new block should be incremental to the past pairwise ordering effort. This pairwise ordering scheme essentially allows SPECTRE to decide between two conflicting blocks (i.e., containing transactions that spend the same UXTO).

2) PHANTOM: PHANTOM [134] is a concurrent blockDAG proposal of SPECTRE by the same authors. One notable weakness of SPECTRE is that the pairwise ordering of blocks may not extend to a fully linear ordering. This leads to SPECTRE's weak liveness (i.e., only supports naturally chronological transactions) and an increased risk of balancing attacks in that conflicts may not be solved. In contrast, PHANTOM realizes fully linear ordering of transactions and blocks in the DAG via the following algorithm. Every node searches the blockDAG for the largest *k*-cluster of blocks. *k* denotes the node degree in the cluster, and is a predefined security parameter that rarely *k* or more honest blocks are created simultaneously. The *k*-cluster is regarded as honest and all blocks within are linearly ordered. The transactions covered by the cluster are then validated in the new order. Notably, the largest 0-cluster case is equivalent to the longest-chain rule of Nakamoto consensus. The authors also argues that PHANTOM can be utilized alongside SPECTRE for more flexible consensus performance, as the two ordering schemes are complimentary to each other.

Security Analysis: Though with an innovative blockDAG ledger structure and a carefully designed ordering scheme for finality, SPECTRE and PHANTOM inherit other protocol components from Nakamoto's, including PoW-based block proposal, gossip-based information propagation, validity check on PoW and transactions. Therefore, the two blockDAG based schemes can tolerate up to 50% of maliciously controlled computing power. On the performance side, blockDAG-based

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

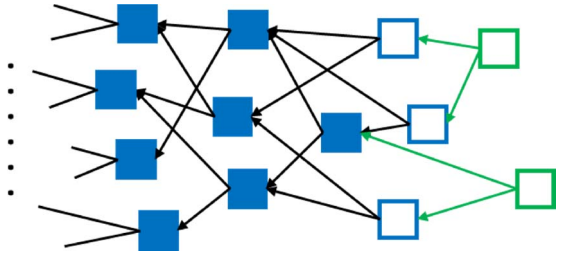


Fig. 10. An example of IOTA Tangle's DAG structure. Every new transaction (hollow green) needs to approve two unapproved transactions ("tips", hollow blue) and becomes a new tip. If only one tip is available, choose an approved transaction for the other. The arrow direction symbolizes a user's tip selection behavior.

schemes can theoretically support arbitrary throughput capacity, only to be capped by network bandwidth and nodes' processing speed. On the downside, the increased parent-selection flexibility may expose more attack surfaces to adaptive attackers (such as the balancing attack against SPECTRE), which is still an ongoing research topic. Alternatively, BlockDAG can be designed to incorporate a specific blockchain as a main chain, reflecting a similar idea as the GHOST rule. Conflux, a blockDAG scheme recently proposed by Li *et al.* [86], resorts to the GHOST rule for the finalization of a *pivot chain*, which is used as the reference for partitioning the blockDAG into chronological order. This scheme yields high transaction throughput but also higher confirmation latency.

G. TxDAG-Based Consensus Protocols

Compared to blockDAG, txDAG further breaks the shackles of block. Since different blocks in a blockDAG can still hold overlapped transactions, it takes extra bandwidth and processing power to solve conflicts. In txDAG, every vertex represents a unique transaction and diverging branches from a vertex shall hold disjoint transactions. This conveniently avoids the conflict resolution effort as in blockDAG and effectively frees up more processing power at each node and allows for higher transaction throughput, only to be capped by network bandwidth. On the other hand, the blockless design of txDAG also leads to more challenges especially regarding to the convergence and effective management of ledger. Next we introduce three txDAG schemes: IOTA Tangle, Byteball, and Nano.

1) IOTA Tangle: IOTA is a blockchain initiative designed for machine-to-machine micro payments in the IoT setting. Tangle, its underlying consensus protocol, was formalized by Popov [135] in 2016. An example of Tangle's DAG is shown in Fig. 10. To append a new transaction to the DAG, a user needs to approve (i.e., validate) two unapproved transactions (tips) of the DAG in order to submit a new transaction. If there is only one tip available, the user chooses an approved transaction instead. The user attaches the hashes of the two chosen transactions to the new transaction and works on a PoW puzzle. The proof is broadcast along with the new transaction. Therefore, every vertex in the DAG has an out-degree of two and contains a PoW proof. The two-tip rule also serves as the incentive mechanism for honest participation. For this reason, IOTA Tangle does not reward block miners with cryptocurrency nor charge transaction fees.

When multiple conflicting tips (i.e., double-spending) are detected by a user, only one can be valid. In this case the user resorts to a tip-selection scheme to choose one that yields the highest acceptance probability in the long term. Other conflicting tips are considered invalid and then orphaned. Tangle currently employs a Markov-chain Monte Carlo (MCMC) based scheme to estimate the long-term acceptance probabilities of conflicting tips.

When there are more than two non-conflicting tips available (i.e., multiple valid choices), the user chooses two tips according to a weight based strategy, which works as follows. Every transaction is assigned an initial weight, which is proportional (discrete values) to the PoW effort spent on this transaction. The user chooses two tips with the highest weights. If the user only possesses a subgraph of the DAG, it performs two weighted random walks from the beginning of the subgraph to decide two tip choices [136]. The weights of a tip transaction and its preceding transactions along the lineage of approvals in the DAG shall accumulate by the weight of the approving transaction. As a result, as time passes the DAG ledger will look like a cascade of transactions that advances toward the direction of the highest accumulative weight. The MCMC-based scheme, the highest-accumulated-weight strategy, and the weighted random walk algorithm can be regarded as the DAG version of longest-chain rule, and essentially contribute to Tangle's tamper-resistance and probabilistic finality.

Theoretically, Tangle's DAG data structure can boost IOTA's transaction capacity to millions per second, or unbounded. This is due to the DAG's native support for branches: when multiple valid tips are present, a new transaction just chooses two to approve, while leaving the remaining tips to other new transactions. As a result, the more new transactions are generated, the more likely a tip gets approved, which can accommodate more new transactions. In practice, Tangle's transaction capacity is still capped by link-/physical-layer communication bandwidth.

2) Byteball: Byteball was proposed by Churyumov [137] in 2016 and currently used in the Obyte cryptocurrency. Byteball features three major differences from Tangle: arbitrary tip number, a main chain (MC) for tip selection, a fixed witness group for DAG consensus and finalization. Every vertex v is assigned a main chain index (MCI), which refers to the height of the closest MC vertex that has a directed path to v. Whenever a node wants to attach a new transaction to a tip, the tip should be chosen based on its validity and MCI. Meanwhile, the MC is determined by the collective effort of the witness group and the network. Byteball's witness group consisting of 12 reputable entities decides the next vertex of MC. The participation of witnesses in the blockDAG helps assign a witness level to any candidate vertex. The one with the higher witness index is attached to MC.

To mitigate Sybil attacks and spams, Byteball collects a storage fee for every byte of transaction content (both header and payload) recorded in the txDAG. These fees, also known as commissions, are paid to witnesses for their honest contribution. Therefore, the witnesses are essentially the consensus

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

participants that work towards Byteball's operation safety. Their revealed identities and small population contributes to deterministic finality and efficient consensus, but also an increased risk of targeted DoS attack.

3) Nano: Nano was proposed by LeMahieu in 2014, formally presented in 2018 [138], and is currently used by the namesake cryptocurrency. Compared to Tangle and Byteball's txDAG ledger, Nano has a more refined ledger structure which is built upon parallel blockchains. Nano's txDAG can be characterized as a block-lattice structure. Every node runs a local blockchain that is not entangled with the others. Notably, we call it "blockchain" for the consistency with the whitepaper [138], though every block is actually a single transaction. A normal transaction is fulfilled by two transactions: a *send* transaction at the sender's blockchain and a *receive* transaction at the receiver's blockchain. PoW is attached to transactions for anti-spamming. When an account is detected double-spending a transaction received by another node (i.e., a fork), the receiver initiates a voting procedure wherein each vote is weighted by the voter's account balance (i.e., stake).

A key insight of Nano's block-lattice based txDAG is that forks and other faulty transactions only affect the accounts referenced in the said transactions. In other words, the resolution of forks does not hamper the processing of transactions from unaffected accounts. This property allows Nano to provide stable and fast transaction confirmation. On the downside, the stake-weighted voting scheme requires high synchrony among nodes and may not scale well in network size.

Security Analysis: For Tangle, since PoW is enforced for transaction generation, probabilistic finality is guaranteed as long as the honest nodes control more than 50% of gross computing power. Byteball's consensus safety relies on the majority consensus of the 12 witnesses. Since Nano resorts to a stake-weighted scheme for fork resolution, its fault tolerance is capped by 50% of tokens. In terms of network decentralization, we consider Tangle the most desirable design among the three txDAG schemes because it does not require a dedicated identified group or a synchronized voting mechanism to ensure consensus finality. Nonetheless, the current version of Tangle's MCMC-based tip-selection is susceptible to parasite chain attack, as is documented in [135]. An attacker can grow a parallel chain from some early point all the way to the current DAG height, with intermittent connections to the DAG. An attacker may need significantly less than 50% of network computing power to succeed, as the malicious chain may carry far fewer transactions than the main DAG. To address this issue, Tangle currently relies on a centralized checkpointing scheme to periodically confirm past transactions. The IOTA community plans to improve the MCMC-based tip-selection scheme in hope of obtaining the 50% fault tolerance in a truly decentralized fashion.

Snowflake-Avalanche [139], a consensus protocol family proposed by Cornell Team Rocket, provides an alternative to Tangle's tip selection problem. Specially, the txDAG-based Avalanche protocol builds upon a hierarchy of leaderless BFT schemes, namely Snowflake and Snowball, for conflict resolution during parent transaction selection. Interested readers are referred to the whitepaper [139] for more details.

VIII. Comparison and Discussion

Tables III, IV compare the protocols that we have discussed so far from three aspects: five-component framework composition, fault tolerance, and transaction processing capability. The latter includes maximum throughput and transaction confirmation latency. For some consensus protocols, not all the five components are specified in the white paper. Protocols designed for permissioned blockchains (e.g., BFT-SMR, PoET, PoTS) often do not specify incentive mechanism; protocols that were initially proposed to substitute PoW (e.g., PoA, PoET, PoR) inherit other components from Nakamoto's protocol by default.

The fault tolerance capability of a consensus protocol largely depends on the block finalization mechanism. For example, the 50% bound (of computing power or stake value) applies to all protocols that inherit the probabilistic finality property from Bitcoin; protocols with BFT-style block finalization typically achieve 33% fault tolerance and guarantee deterministic finality. In hybrid schemes, the 33% fault tolerance is usually coupled with the block proposal mechanism or a preexisting identity scheme. In Hybrid PoW-BFT and BFT-based PoS, each node's voting weight in the BFT consensus stage is augmented by its block proposing capability. This is achieved by either linking a player's opportunity of participation in BFT consensus to the PoX process (e.g., Hybrid PoW-BFT, Tendermint), or directly weighting a player's vote in the BFT consensus by the player's stake value (e.g., Algorand, Casper FFG). In other hybrid schemes such as PoA-BFT and PoET-BFT, the PoX process is purposed for identity management and thus the fault tolerance threshold is 33% of identities. DAG-based protocols generally achieves probabilistic finality (except Byteball) and 50% fault tolerance of either computing power, consensus participants, or token wealth.

The statistics on transaction processing capability were fetched from either the simulation result in protocol white paper or documented experiment. The throughput metric measures the maximum TPS a protocol can handle, and is only precise to the scale of magnitude, i.e., sub-ten, tens, hundreds, or thousands. The confirmation latency metric estimates time for a submitted transaction to be finalized in the blockchain. We observe that protocols with probabilistic finality tend to have higher confirmation latency (typically more than 1 minute), and high throughput is often accompanied by low confirmation latency.

We are also interested in another performance metric—scalability in network size, which measures how well a consensus protocol can maintain its transaction capacity when network size grows. Given the difficulty of performing large-scale experiments on actual blockchain networks, the scalability results are either fetched from the simulation study in protocol white paper or based on our speculation on the current network status. It is worth noting that transaction throughput and scalability in network size are often recognized as two integral parts in scalability evaluation of a blockchain system [3], [18], [140].

Fig. 11 illustrates performance of consensus protocols with respect to transaction throughput and scalability in network size along with suitability for permissioned or permissionless

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

TABLE III
A COMPARISON OF BLOCKCHAIN CONSENSUS PROTOCOLS

Consensus Protocol (Examples/ Realizations)	**Five-Component Framework**					**Fault Tolerance**	**Throughput** (TPS) **[Confirmation Latency]**
	Block Proposal	**Block Validation***	**Information Propagation**	**Block Finalization**	**Incentive Mechanism**		
BFT-SMR (PBFT, Honey-BadgerBFT)	Client operation request	Signature check	Broadcast	Mutual agreement on the same state	(N/A)	33% servers	Thousands [1s-1m]
Nakamoto (Bitcoin, Litecoin)	PoW	PoW check	Gossiping	Longest-chain rule†	Block reward and transaction fee	50% computing power	Sub-ten [10m-60m]
Nakamoto-GHOST (Ethereum)	PoW (Ethash)	PoW check	Gossiping via secure channels	A variation of GHOST rule†	Block reward and transaction fee	50% computing power	Tens [6m-10m]
Bitcoin-NG (Waves-NG)	PoW for key blocks	PoW check for key blocks	Gossiping	Longest-chain rule†	Block reward and transaction fee	50% computing power	Hundreds [10m]
Hybrid PoW-BFT (PeerConsens, SCP, ByzCoin)	PoW-based BFT committee election	PoW check	Broadcast among BFT committee	Longest-chain rule† for chain (PBFT for transactions)	⊙	33% computing power	Hundreds [10s-1m]
Chain-based PoS (Peercoin, Nxt, PoAct)	PoS	PoS check	Gossiping	Longest-chain rule†	Block reward and transaction fee	50% deposited stake value	Tens [10-60m]
Committee-based PoS (Ouroboros, Praos, CoA, Snow White)	PoS-based committee election	Proposer eligibility check	Broadcast among committee	Longest-chain rule‡	Block reward	50% token wealth	Hundreds [1m-10m]
BFT-based PoS —Tendermint (Cosmos Hub)	PoS-based round robin	Proposer eligibility check	Broadcast among validators	BFT (adapted DLS)	Block reward	33% token wealth	Thousands [1s-3s]
BFT-based PoS —Algorand	PoS-based committee election	Proposer eligibility check	Broadcast among committee	BFT (adapted Byzantine agreement)	Block reward	33% token wealth	Hundreds [20s-1m]
BFT-based PoS —Casper FFG (Ethereum 2.0)	PoW (Ethash)	PoW & Checkpoint tree check	Broadcast among validators	BFT (with staked votes)	Block reward for miners and validators	33% deposited stake value	Thousands (if sharding used)
DPoS (EOSIO, Lisk, BitShares)	PoS with stake delegation	Delegate eligibility check	Broadcast among delegates	BFT (suggested, not limited to)	Block reward and vote reward	33% delegates	Thousands [$<$1s]
PoA (Rinkby, Kovan, POA Network)	PoA	Block proposer identity check	⊙ Broadcast‡	⊙ HoneyBadgerBFT‡	⊙ Transaction fee‡	50% TEEs (33% if BFT used)	Tens to hundreds [10s-1m]
PoET (Hyperledger Sawtooth family)	PoET within TEE	TEE certificate check	⊙ Broadcast‡	⊙ PBFT‡	⊙	50% IDs (33% if BFT used)	Tens to thousands [10s-1m]
PoTS	PoTS-based committe election	Proposer eligibility&TEE cert. check	⊙	⊙	⊙	50% token wealth	(Unavailable)
PoR (Permacoin)	PoR	File retrievability check	⊙	⊙	⊙	50% storage space	(Unavailable)
RPCA (Ripple)	Any server can propose transactions	UNL membership check	Broadcast to UNL peers	Accepting $>$ 80% voted transactions	Transaction fee	20% nodes in each UNL	Thousands [$<$1s]

*Block validation also includes validating all transactions inside the block, which is omitted here for space saving.
†: With probabilistic finality. ‡: Unspecified in the protocol white paper, but found in one or more blockchain realizations. ⊙: Unspecified in the protocol white paper, but the Bitcoin counterpart is usable.

blockchains. For protocols with BFT-style block finalization mechanism to achieve thousands of TPS, the network size should be small, typically around several hundred. Meanwhile, protocols that work for large-scale public networks are predominantly permissionless and employ a block finalization mechanism that only achieves probabilistic finality. Their transaction throughput is usually capped by a hundred TPS for security reasons. Generally, protocols illustrated farther to the top right in Fig. 11 tend to achieve better overall performance and scalability. However the security implication

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
A COMPARISON OF DAG-BASED CONSENSUS PROTOCOLS

Consensus Protocol (Blockchain Realizations)	**Five-Component Framework**					**Fault Tolerance**	**Throughput** (TPS) **[Confirmation Latency]**
	Block/TX Proposal	**Block/TX Validation**	**Information Propagation**	**Block/TX Finalization**	**Incentive Mechanism**		
BlockDAG —SPECTRE —PHANTOM	PoW	PoW check	Gossiping	Pairwise ordering†/ *k*-cluster-blocks ordering†	⊙	50% computing power	Thousands or more [10s-2m]
BlockDAG —Conflux	PoW	PoW check	Gossiping	GHOST rule to finalize pivot chain†	⊙	50% computing power	Thousands [5m-20m]
TxDAG —Tangle (IOTA)	Approving two tips & PoW	Tip approval check & PoW check	Gossiping	Highest cumulative weight rule†	Eligibility for issuing new transactions	50% computing power	Thousands or more [1m-10m]
TxDAG —Byteball (Obyte)	Approving tips per main chain index	Tip approval check	Gossiping	Witnesses consensus on main chain	Commissions collected from storage fees	50% (or 6) witnesseses	Thousands or more [30s-10m]
TxDAG —Nano (Nano)	Cross-chain sender-receiver & PoW	Sender account & PoW check	Gossiping	stake-weighted voting†	(Unspecified)	50% token wealth	Thousands or more [1s-10s]
TxDAG —Avalanche	Approving one parent	Parent approval check	Gossiping	Confidence ordering to solve conflict†	(Unspecified)	50% participants	Thousands [1s-5s]

†: With probabilistic finality. ⊙: Unspecified in the protocol white paper, but the Bitcoin counterpart is usable.

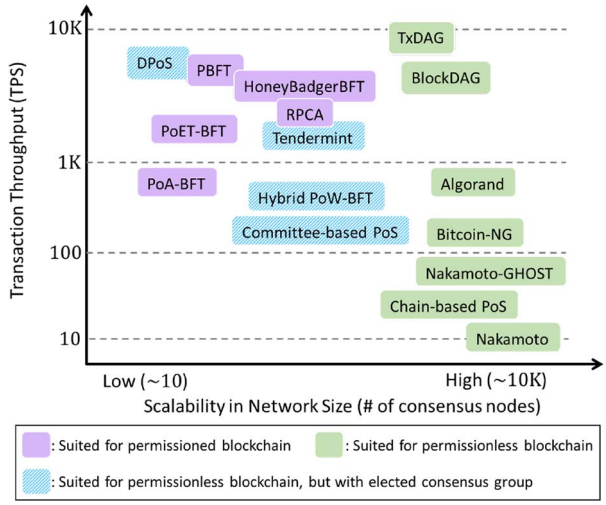


Fig. 11. Comparing different blockchain consensus protocols on transaction throughput and scalability in network size. Note that for comparison purposes, we consider DAG-based schemes as wide-sense blockchain protocols.

of such superiority, exemplified by Bitcoin-NG and DAG-based protocols, is subject to constant scepticism and attracts ongoing research effort.

As for the suitability for permissioned or permissionless blockchains, we stress that protocols that need a stable consensus group with participation control and identification are suitable for permissioned blockchains. These protocols include PBFT, HoneyBadgerBFT, PoET-BFT, PoA-BFT, and RPCA. In comparison, protocols including Nakamoto, Nakamoto-GHOST, chain-based PoS, Bitcoin-NG, Algorand, and Tangle are specifically designed for large-scale permissionless blockchains. Meanwhile DPoS, Tendermint, hybrid PoW-BFT, committee-based PoS are designed for permissionless scenarios but rely on an election mechanism for maintaining a stable consensus group, of which the identities may be revealed for public supervision. DAG-based protocols are designed with the aim of scaling up in participant count and thus suited for permissionless networks.

IX. ON DESIGNING BLOCKCHAIN CONSENSUS PROTOCOL

The rich variety of blockchain initiatives often confounds novice protocol designers. In this section we offer a succinct tutorial that hopefully helps protocol designers form reasonable objectives and avoid pitfalls.

A. The Paradigm Shift in Protocol Design

In the early days of blockchain development, consensus protocol designs were often coupled with designer's ingenuity and heuristics. Anticipating that the increasing number of participants would lead to more block contentions and blockchain forks in Bitcoin, Nakamoto designated the longest-chain rule for block finalization and cautiously fixed the average block interval to ten minutes for consensus security [1]. Litecoin [72] and Ethereum [71] adopted shorter block intervals for faster transaction settlement. Observing the energy inefficiency of Bitcoin and the financial value of unused tokens, Peercoin pioneered the PoS mechanism which could substitute PoW with far less energy consumption [6]. Yet its stake valuation scheme seemed to be heuristically designed. These early-day initiatives also commonly lacked sufficient security analysis, which were often scrutinized by researchers and practitioners [74], [79], [96].

Later blockchain consensus protocols, especially those proposed after 2016, have started to take inspirations from the

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

established research of distributed computing, cryptography, and trusted computing. For example, HoneyBadgerBFT [16], BEAT [17], Algorand [13] extend reliable broadcast and Byzantine agreement to blockchain scenarios. Committee-based PoS protocols, such as Ouroboros [8], Snow White [9], Bentov's CoA [100], make use of MPC for the management of consensus committee and block proposal. PoET [119] and PoTS [127] leverage trusted execution environment (TEE) for secure block proposal and mining substitution. Furthermore, some proposals incorporate innovative cryptographic primitives for enhancing privacy of network participants. Zero-knowledge proof (ZKP) and ring signatures are used by Zerocash [141] and Monero [142] to construct privacy-preserving transactions that hide essential transaction values and sender-receiver addresses; Algorand and Ouroboros Praos [101] use cryptographic sortition and VRF for the PoS-based election of consensus committee without revealing participant's stake value. The privacy of consensus participants in large-scale permissionless blockchains is still a fresh research topic.

Besides increased sophistication in design, new protocol proposals are often accompanied by formal security analysis, via security frameworks such as the *random oracle model* [143] and *universal composibility* [144]. In summary, This paradigm shift has brought the design and analysis of blockchain consensus protocols into formal scientific research, echoing Cachin and Vukolić [19] that blockchain design should follow the rigor established by prevailing wisdom.

B. The Security–Decentralization–Scalability Trilemma

Observing the existence of various blockchain consensus protocols, we stress that the design of consensus protocol should seek a balance between three objectives: security, decentralization, and scalability.

- *Security:* refers to the blockchain's consensus security in the presence of malicious players and anti-censorship capability. The two concepts correspond to the safety and liveness property in classical distributed consensus.
- *Decentralization:* refers to the decentralization profile of the blockchain network, which is normally associated with geographic diversity, connectivity pattern, and synchrony of networked nodes.
- *Scalability:* means two-fold: the system needs to remain secure and efficient with rising transaction throughput as well as larger network size. Scalability is often considered as a broader concept of system performance.

Fig. 12 sketches the relationships and trade-offs between the three objectives.

For most blockchain consensus protocols especially those tailored for financial application, security is always the top priority. Meanwhile, security level can be practically traded for system's scalability. Lower security requirement gives rise to more scalable protocol design. For classical BFT protocols, lower fault tolerance threshold (the "f" parameter) leads to fewer consensus rounds [25], [55] or less communication per round [27], effectively alleviating message complexity caused by large network sizes. For many public blockchain networks

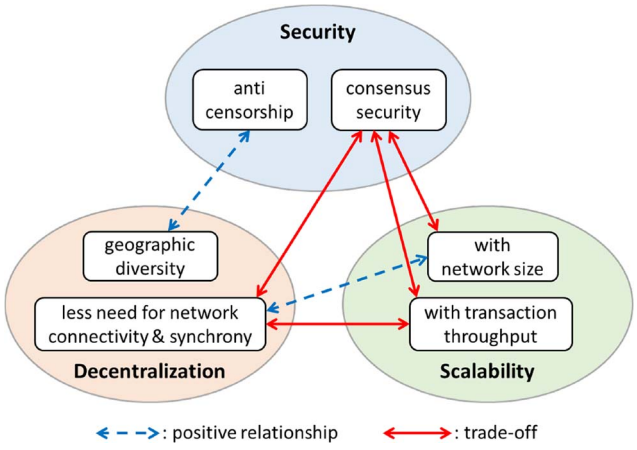


Fig. 12. Illustration of the security-decentralization-scalability trilemma.

that use Nakamoto consensus, the probabilistic finality and mandatory multi-block confirmation give rise to permissionless access and higher scalability in network size. Meanwhile, a shorter block interval yields higher transaction throughput but also leads to higher risk of 51%-attack [73]. For DAG-based protocols, the security implications associated with their scalability with both network size and throughput are not well studied and remain an ongoing research.

A protocol designer should anticipate the security implications of a blockchain network's decentralization profile. A more decentralized network tends to be less synchronized, owing to the increased diversity of geographic locations and sparsity of connections. On one hand, the increased geographic diversity makes censorship by one suppressing regime more challenging, which enhances the system's liveness. On the other hand, the reduced synchrony implies that inter-player communications are less bounded by delay requirements. This contributes to the heterogeneity of network connections and allows better connected players to gain unfair advantages, which potentially impairs consensus security of the network. Take protocols with the gossiping rule and longest-chain rule for example, the heterogeneity of network connections gives well-connected nodes a communication advantage of disseminating new blocks faster in the network than the less-connected ones, effectively enhancing the former's winning chance in blockchain fork races. As a result, in a sparsely connected network with one well-connected adversary, the adversary may need significantly less than 50% of mining power to commit a double-spending attack.

Last but not least, a protocol designer should be aware that the tradeoff between decentralization and scalability depends on the practical needs. For example, Bitcoin was designed to operate in a weakly synchronized network and there is no enforceable criterion for message delivery and timeout. As a result Nakamoto consensus protocol needs to rely on best-effort mining and long block intervals to achieve the security against double spending, at the price of low transaction throughput. In comparison, BFT-based protocols are designed for permissioned networks where every participant operates with revealed identity and the overall network is

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

small and well-connected and with enforceable time-out mechanisms. This enables the network to achieve higher transaction throughput. On the flip side, the high dependence on network connectivity and synchony for throughput brings high messaging complexity and network management overhead. When the network grows in size and inevitably becomes sparse, message relaying can be a major burden for each participant which potentially leads to jammed communication channel and stalled consensus process. Therefore, we argue that under a certain security premise, increased need for network connectivity and synchrony leads to higher transaction throughput but lower scalability with network size.

C. Protocol Composability

The discussions above demonstrate that there is never a one-fits-all blockchain consensus protocol. The existence of hybrid consensus protocols highlights the possibility to cherry-pick individual protocol components to fulfill specific application needs. Sawtooth [119], a Hyperledger project featuring the utilization of trusted hardware for PoET-based block proposal, has the flexibility of plugging in different block finalization schemes—Raft for crash fault tolerance and PBFT for Byzantine fault tolerance. Similarly, POA Network [11] uses PoA for block proposal and has opted for HoneyBadgerBFT for block finalization to achieve best performance under asynchronous network conditions.

Besides block proposal and finalization, the choice of information propagation mechanism largely depends on the underlying network topology. In a public blockchain network where gossiping is the default information propagation method, broadcast can be used among a small set of identified nodes for better efficiency [3]. The block validation mechanism is associated with the block proposal mechanism and takes the current blockchain as input. The incentive mechanism is aimed for promoting honest execution of the former tasks. It needs to account for off-chain factors as well, such as the long-term system sustainability, financial stability, and subtle security issues [115], [116], [117].

The composability of blockchain consensus protocol also demonstrates the feasibility of progressive protocol improvement, in that one individual protocol component is updated at a time instead of overhauling the entire protocol. This is particularly important for large-scale established blockchain systems in which any protocol change requires network-wide consensus and significant joint effort.

X. Conclusion

In this survey we provided a summary of classical fault-tolerance consensus research, a five-component framework for a general blockchain consensus protocol, and a comprehensive review of blockchain consensus protocols that have gained great popularity and potential. We analyzed these protocols with respect to fault tolerance, performance, vulnerabilities and highlighted their use cases. Notably, many of these protocols are still under development and are subject to major changes at the time of writing. We hope the five-component framework, classification methodology, protocol abstractions, performance analyses, and discussion on protocol design provided in this survey can help researchers and developers grasp the fundamentals of blockchain consensus protocols and facilitate future protocol design.

Acknowledgment

The authors would like to thank Dr. Christian Cachin, Dr. Yonggang Wen for their constructive comments on this paper.

References

[1] S. Nakamoto. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Accessed: Apr. 7, 2019. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] *Global Bitcoin Nodes Distribution*. Accessed: Apr. 7, 2019. [Online]. Available: https://bitnodes.earn.com/

[3] K. Croman *et al.*, "On scaling decentralized blockchains," in *Proc. Int. Conf. Financial Cryptography Data Security*, 2016, pp. 106–125.

[4] *VISA Fact Sheet*, VISA Inc., San Francisco, CA, USA, 2019. [Online]. Available: https://usa.visa.com/dam/VCOM/global/about-visa/documents/visa-fact-sheet-july-2019.pdf

[5] *Bitcoin Energy Consumption Index, Digiconomist*. Accessed: Nov. 29, 2019. [Online]. Available: https://digiconomist.net/bitcoin-energy-consumption

[6] S. King and S. Nadal. (Aug. 2012). *PPcoin: Peer-to-Peer Crypto-Currency With Proof-of-Stake*. [Online]. Available: https://bitcoin.peryaudo.org/vendor/peercoin-paper.pdf

[7] I. Eyal, A. E. Gencer, E. G. Sirer, and R. Van Renesse, "Bitcoin-NG: A scalable blockchain protocol," in *Proc. NSDI*, 2016, pp. 45–59.

[8] A. Kiayias, A. Russell, B. David, and R. Oliynykov, "Ouroboros: A provably secure proof-of-stake blockchain protocol," in *Proc. Annu. Int. Cryptol. Conf.*, 2017, pp. 357–388.

[9] P. Daian, R. Pass, and E. Shi, "Snow white: Robustly reconfigurable consensus and applications to provably secure proof of stake," in *Proc. Int. Conf. Financ. Cryptography Data Security*, 2019, pp. 23–41.

[10] (2018). *EOS.IO Technical White Paper V2*. [Online]. Available: https://github.com/EOSIO/Documentation/blob/master/TechnicalWhitePaper.md

[11] (2018). *POA Network Whitepaper*. [Online]. Available: https://github.com/poanetwork/wiki/wiki/POA-Network-Whitepaper

[12] J. Y. Kwon. (2014). *TenderMint: Consensus Without Mining*. [Online]. Available: https://cdn.relayto.com/media/files/LPgoWO18TCeMIggJVakt_tendermint.pdf

[13] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, "Algorand: Scaling Byzantine agreements for cryptocurrencies," in *Proc. ACM 26th Symp. Oper. Syst. Principles*, 2017, pp. 51–68.

[14] V. Buterin and V. Griffith, "Casper the friendly finality gadget," 2017. [Online]. Available: arXiv:1710.09437.

[15] E. Androulaki *et al.*, "Hyperledger fabric: A distributed operating system for permissioned blockchains," in *Proc. ACM 13th EuroSys Conf.*, 2018, p. 30.

[16] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song, "The honey badger of BFT protocols," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2016, pp. 31–42.

[17] S. Duan, M. K. Reiter, and H. Zhang, "BEAT: Asynchronous BFT made practical," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2018, pp. 2028–2041.

[18] M. Vukolić, "The quest for scalable blockchain fabric: Proof-of-work vs. BFT replication," in *Proc. Int. Workshop Open Problems Netw. Security*, 2015, pp. 112–125.

[19] C. Cachin and M. Vukolić, "Blockchain consensus protocols in the wild," in *Proc. 31st Int. Symp. Distrib. Comput. (DISC)*, 2017, pp. 1–16.

[20] S. Bano *et al.*, "Consensus in the age of blockchains," 2017. [Online]. Available: arXiv:1711.03936.

[21] W. Wang *et al.*, "A survey on consensus mechanisms and mining strategy management in blockchain networks," *IEEE Access*, vol. 7, pp. 22328–22370, 2019.

[22] Y. Xiao, N. Zhang, J. Li, W. Lou, and Y. T. Hou, "Distributed consensus protocols and algorithms," in *Blockchain for Distributed Systems Security*. Hoboken, NJ, USA: Wiley, 2019, p. 25.

[23] M. Belotti, N. Božić, G. Pujolle, and S. Secci, "A vademecum on blockchain technologies: When, which and how," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 4, pp. 3796–3838, 4th Quart., 2019.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

[24] H. Attiya, A. Bar-Noy, and D. Dolev, "Sharing memory robustly in message-passing systems," *J. ACM*, vol. 42, no. 1, pp. 124–142, 1995.

[25] L. Lamport, R. Shostak, and M. Pease, "The Byzantine generals problem," *ACM Trans. Program. Lang. Syst.*, vol. 4, no. 3, pp. 382–401, 1982.

[26] H. Attiya and J. Welch, *Distributed Computing: Fundamentals, Simulations, and Advanced Topics*, vol. 19. Hoboken, NJ, USA: Wiley, 2004.

[27] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *Proc. OSDI*, vol. 99, 1999, pp. 173–186.

[28] C. Dwork, N. A. Lynch, and L. J. Stockmeyer, "Consensus in the presence of partial synchrony," *J. ACM*, vol. 35, no. 2, pp. 288–323, 1988.

[29] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *J. ACM*, vol. 32, no. 2, pp. 374–382, 1985.

[30] M. Pease, R. Shostak, and L. Lamport, "Reaching agreement in the presence of faults," *J. ACM*, vol. 27, no. 2, pp. 228–234, 1980.

[31] B. Alpern and F. B. Schneider, "Defining liveness," *Inf. Process. Lett.*, vol. 21, no. 4, pp. 181–185, 1985.

[32] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[33] L. Lamport, "Using time instead of timeout for fault-tolerant distributed systems," *ACM Trans. Program. Lang. Syst.*, vol. 6, no. 2, pp. 254–280, 1984.

[34] F. B. Schneider, "Implementing fault-tolerant services using the state machine approach: A tutorial," *ACM Comput. Surveys*, vol. 22, no. 4, pp. 299–319, 1990.

[35] F. Cristian, H. Aghili, R. Strong, and D. Dolev, *Atomic Broadcast: From Simple Message Diffusion to Byzantine Agreement*, T. J. Watson, Ed. Yorktown Heights, NY, USA: Int. Bus. Mach., 1986.

[36] G. Bracha and S. Toueg, "Asynchronous consensus and broadcast protocols," *J. ACM*, vol. 32, no. 4, pp. 824–840, 1985.

[37] X. Défago, A. Schiper, and P. Urbán, "Total order broadcast and multicast algorithms: Taxonomy and survey," *ACM Comput. Surveys*, vol. 36, no. 4, pp. 372–421, 2004.

[38] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," *J. ACM*, vol. 43, no. 2, pp. 225–267, 1996.

[39] B. M. Oki and B. H. Liskov, "Viewstamped replication: A new primary copy method to support highly-available distributed systems," in *Proc. 7th Annu. ACM Symp. Principles Distrib. Comput.*, 1988, pp. 8–17.

[40] B. Liskov and J. Cowling, "Viewstamped replication revisited," Comput. Sci. Artif. Intell. Lab., MIT, Cambridge, MA, USA, Rep. MIT-CSAIL-TR-2012-021, 2012.

[41] L. Lamport, "The part-time parliament," *ACM Trans. Comput. Syst.*, vol. 16, no. 2, pp. 133–169, 1998.

[42] L. Lamport, "Paxos made simple," *ACM SIGACT News*, vol. 32, no. 4, pp. 18–25, 2001.

[43] D. Ongaro and J. K. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2014, pp. 305–319.

[44] M. Abd-El-Malek, G. R. Ganger, G. R. Goodson, M. K. Reiter, and J. J. Wylie, "Fault-scalable Byzantine fault-tolerant services," *ACM SIGOPS Oper. Syst. Rev.*, vol. 39, no. 5, pp. 59–74, 2005.

[45] J. A. Cowling, D. S. Myers, B. Liskov, R. Rodrigues, and L. Shrira, "HQ replication: A hybrid quorum protocol for Byzantine fault tolerance," in *Proc. 7th Symp. Oper. Syst. Design Implement.*, 2006, pp. 177–190.

[46] R. Kotla, L. Alvisi, M. Dahlin, A. Clement, and E. Wong, "Zyzzyva: Speculative Byzantine fault tolerance," *ACM SIGOPS Oper. Syst. Rev.*, vol. 41, no. 6, pp. 45–58, 2007.

[47] J.-P. Martin and L. Alvisi, "Fast Byzantine consensus," *IEEE Trans. Depend. Secure Comput.*, vol. 3, no. 3, pp. 202–215, Aug. 2006.

[48] G. S. Veronese, M. Correia, A. N. Bessani, and L. C. Lung, "Spin one's wheels? Byzantine fault tolerance with a spinning primary," in *Proc. 28th IEEE Int. Symp. Rel. Distrib. Syst. (SRDS)*, 2009, pp. 135–144.

[49] A. Clement, E. L. Wong, L. Alvisi, M. Dahlin, and M. Marchetti, "Making Byzantine fault tolerant systems tolerate Byzantine faults," in *Proc. NSDI*, vol. 9, 2009, pp. 153–168.

[50] R. Guerraoui, N. Knežević, V. Quéma, and M. Vukolić, "The next 700 BFT protocols," in *Proc. 5th Eur. Conf. Comput. Syst.*, 2010, pp. 363–376.

[51] A. N. Bessani, "(BFT) state machine replication: The hype, the virtue, and even some practice," in *Proc. EuroSys*, 2012, pp. 22–49. [Online]. Available: http://www.di.fc.ul.pt/∼bessani/publications/T1-bftsmr.pdf

[52] G. Bracha, "Asynchronous Byzantine agreement protocols," *Inf. Comput.*, vol. 75, no. 2, pp. 130–143, 1987.

[53] M. Ben-Or, "Another advantage of free choice (extended abstract): Completely asynchronous agreement protocols," in *Proc. 2nd Annu. ACM Symp. Principles Distrib. Comput.*, 1983, pp. 27–30.

[54] M. O. Rabin, "Randomized Byzantine generals," in *Proc. 24th Annu. Symp. Found. Comput. Sci. (SFCS)*, 1983, pp. 403–409.

[55] G. Bracha, "An asynchronous $[(n-1)/3]$-resilient consensus protocol," in *Proc. 3rd Annu. ACM Symp. Principles Distrib. Comput.*, 1984, pp. 154–162.

[56] M. Ben-Or, B. Kelmer, and T. Rabin, "Asynchronous secure computations with optimal resilience," in *Proc. 13th Annu. ACM Symp. Principles Distrib. Comput.*, 1994, pp. 183–192.

[57] R. Canetti and T. Rabin, "Fast asynchronous Byzantine agreement with optimal resilience," in *Proc. 25th Annu. ACM Symp. Theory Comput.*, 1993, pp. 42–51.

[58] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and efficient asynchronous broadcast protocols," in *Proc. Annu. Int. Cryptol. Conf.*, 2001, pp. 524–541.

[59] C. Cachin and S. Tessaro, "Asynchronous verifiable information dispersal," in *Proc. 24th IEEE Symp. Rel. Distrib. Syst. (SRDS)*, 2005, pp. 191–201.

[60] A. Mostéfaoui, H. Moumen, and M. Raynal, "Signature-free asynchronous Byzantine consensus with $t < n/3$ and $o(n^2)$ messages," in *Proc. ACM Symp. Principles Distrib. Comput.*, 2014, pp. 2–9.

[61] J. Baek and Y. Zheng, "Simple and efficient threshold cryptosystem from the gap Diffie–Hellman group," in *Proc. IEEE Glob. Telecommun. Conf. (GLOBECOM)*, vol. 3, 2003, pp. 1491–1495.

[62] V. Shoup and R. Gennaro, "Securing threshold cryptosystems against chosen ciphertext attack," in *Proc. Int. Conf. Theory Appl. Cryptograph. Techn.*, 1998, pp. 1–16.

[63] J. Yin, J.-P. Martin, A. Venkataramani, L. Alvisi, and M. Dahlin, "Separating agreement from execution for Byzantine fault tolerant services," in *Proc. ACM SIGOPS Oper. Syst. Rev.*, vol. 37, 2003, pp. 253–267.

[64] C. Decker, J. Seidel, and R. Wattenhofer, "Bitcoin meets strong consistency," in *Proc. 17th Int. Conf. Distrib. Comput. Netw.*, 2016, p. 13.

[65] Bitcoin Wire Protocol 101. (2019). *Bitcoin Developer Network*. [Online]. Available: https://bitcoindev.network/bitcoin-wire-protocol/

[66] Ethereum Wire Protocol. (2019). *Ethereum Foundation*. [Online]. Available: https://github.com/ethereum/devp2p/blob/master/caps/eth.md

[67] K. Wüst and A. Gervais, "Do you need a blockchain?" in *Proc. IEEE Crypto Valley Conf. Blockchain Technol. (CVCBT)*, 2018, pp. 45–54.

[68] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. Conf. Theory Appl. Cryptograph. Techn.*, 1987, pp. 369–378.

[69] J. Poon and T. Dryja. (2016). *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments*. Accessed: Apr. 7, 2020. [Online]. Available: https://www.bitcoinlightning.com/wp-content/uploads/2018/03/lightning-network-paper.pdf

[70] EIP 1011: Hybrid Casper FFG. *Ethereum Improvement Proposals (EIPs)*. Accessed: Apr. 7, 2019. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1011

[71] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," vol. 151, Zug, Switzerland, Ethereum, Yellow Paper pp. 1–32, 2014.

[72] Litecoin Official Website. *Litecoin Foundation*. Accessed: Apr. 7, 2019. [Online]. Available: https://litecoin.org/

[73] C. Decker and R. Wattenhofer, "Information propagation in the bitcoin network," in *Proc. IEEE 13th Int. Conf. Peer-to-Peer Computing (P2P)*, 2013, pp. 1–10.

[74] J. A. Garay, A. Kiayias, and N. Leonardos, "The bitcoin backbone protocol: Analysis and applications," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2015, pp. 281–310.

[75] I. Abraham and D. Malkhi, "The blockchain consensus layer and BFT," *Bull. EATCS*, vol. 3, no. 123, pp. 10–17, 2017.

[76] *VisaNet: The Technology Behind Visa, Visa Inc.* Accessed: Apr. 7, 2019. [Online]. Available: https://usa.visa.com/dam/VCOM/download/corporate/media/visanet-technology/visa-net-booklet.pdf

[77] E. Heilman, A. Kendler, A. Zohar, and S. Goldberg, "Eclipse attacks on bitcoin's peer-to-peer network," in *Proc. USENIX Security Symp.*, 2015, pp. 129–144.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

[78] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun, "On the security and performance of proof of work blockchains," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2016, pp. 3–16.

[79] I. Eyal and E. G. Sirer, "Majority is not enough: Bitcoin mining is vulnerable," in *Proc. Int. Conf. Financial Cryptography Data Security*, 2014, pp. 436–454.

[80] A. Sapirshtein, Y. Sompolinsky, and A. Zohar, "Optimal selfish mining strategies in bitcoin," in *Proc. Int. Conf. Financial Cryptography Data Security*, 2016, pp. 515–532.

[81] D. Gutteridge. (2018). *Japanese Cryptocurrency Monacoin Hit by Selfish Mining Attack*. [Online]. Available: https://www.ccn.com/japanese-cryptocurrency-monacoin-hit-by-selfish-mining-attack/

[82] A. E. Gencer, S. Basu, I. Eyal, R. van Renesse, and E. G. Sirer, "Decentralization in bitcoin and Ethereum networks," in *Proc. Financial Cryptography Data Security Conf.*, 2018, pp. 439–457.

[83] D. Kondor, M. Pósfai, I. Csabai, and G. Vattay, "Do the rich get richer? An empirical analysis of the bitcoin transaction network," *PLoS ONE*, vol. 9, no. 2, 2014, Art. no. e86197.

[84] Y. Sompolinsky and A. Zohar, "Secure high-rate transaction processing in bitcoin," in *Proc. Int. Conf. Financial Cryptography Data Security*, 2015, pp. 507–527.

[85] *Wave-NG Protocol, Wave Platform*. Accessed: Oct. 14, 2019. [Online]. Available: https://docs.wavesplatform.com/en/waves-environment/waves-protocol/waves-ng-protocol.html

[86] C. Li, P. Li, W. Xu, F. Long, and A. C.-C. Yao, "Scaling Nakamoto consensus to thousands of transactions per second," 2018. [Online]. Available: arXiv:1805.03870.

[87] M. K. Reiter, "A secure group membership protocol," *IEEE Trans. Softw. Eng.*, vol. 22, no. 1, pp. 31–42, Jan. 1996.

[88] L. Luu, V. Narayanan, K. Baweja, C. Zheng, S. Gilbert, and P. Saxena. (2015). *SCP: A Computationally-Scalable Byzantine Consensus Protocol for Blockchains*. [Online]. Available: https://www.weusecoins.com/assets/pdf/library/SCP

[89] *Sharding FAQs: On Sharding Blockchains, Ethereum Foundation*. Accessed: Oct. 14, 2019. [Online]. Available: https://github.com/ethereum/wiki/wiki/Sharding-FAQ

[90] E. Kokoris-Kogias, P. Jovanovic, N. Gailly, I. Khoffi, L. Gasser, and B. Ford, "Enhancing bitcoin security and performance with strong consistency via collective signing," in *Proc. 25th USENIX Security Symp. (USENIX Security)*, 2016, pp. 279–296.

[91] E. Syta *et al.*, "Keeping authorities 'honest or bust' with decentralized witness cosigning," in *Proc. IEEE Symp. Security Privacy (SP)*, 2016, pp. 526–545.

[92] R. Pass and E. Shi, "Hybrid consensus: Efficient consensus in the permissionless model," in *Proc. 31st Int. Symp. Distrib. Comput. (DISC)*, 2017, pp. 1–16.

[93] R. Pass and E. Shi, "FruitChains: A fair blockchain," in *Proc. ACM Symp. Principles Distrib. Comput.*, 2017, pp. 315–324.

[94] J. Long and R. Wei, "Scalable BFT consensus mechanism through aggregated signature Gossip," in *Proc. IEEE Int. Conf. Blockchain Cryptocurrency (ICBC)*, 2019, pp. 360–367.

[95] A. Poelstra. (2014). *Distributed Consensus From Proof of Stake is Impossible*. Accessed: Apr. 7, 2019. [Online]. Available: https://download.wpsoftware.net/bitcoin/old-pos.pdf

[96] (2014). *NXT Whitepaper Revision 4, NXT Community*. [Online]. Available: https://www.dropbox.com/s/cbuwrorf672c0yy/NxtWhitepaper_v122_rev4.pdf

[97] I. Bentov, C. Lee, A. Mizrahi, and M. Rosenfeld, "Proof of activity: Extending bitcoin's proof of work via proof of stake [extended abstract]y," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 42, no. 3, pp. 34–37, 2014.

[98] T. Rabin and M. Ben-Or, "Verifiable secret sharing and multiparty protocols with honest majority," in *Proc. 21st Annu. ACM Symp. Theory Comput.*, 1989, pp. 73–85.

[99] S. Micali, M. Rabin, and S. Vadhan, "Verifiable random functions," in *Proc. IEEE 40th Annu. Symp. Found. Comput. Sci.*, 1999, pp. 120–130.

[100] I. Bentov, A. Gabizon, and A. Mizrahi, "Cryptocurrencies without proof of work," in *Proc. Int. Conf. Financial Cryptography Data Security*, 2016, pp. 142–157.

[101] B. David, P. Gaži, A. Kiayias, and A. Russell, "Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain," in *Proc. Annu. Int. Conf. Theory Appl. Cryptograph. Techn.*, 2018, pp. 66–98.

[102] R. Pass and E. Shi, "The sleepy model of consensus," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Security*, 2017, pp. 380–409.

[103] E. Buchman, "TenderMint: Byzantine fault tolerance in the age of blockchains," Ph.D. dissertation, Eng. Syst. Comput., Univ. Guelph, Guelph, ON, Canada, 2016.

[104] *Cosmos SDK Documentation—Validators Overview*, Tendermint Inc., San Francisco, CA, USA, 2019. [Online]. Available: https://hub.cosmos.network/master/validators/overview.html

[105] *Tendermint Documentation: Sentry Node*, Tendermint Inc., San Francisco, CA, USA, 2019. Accessed: Apr. 7, 2019. [Online]. Available: https://tendermint.com/docs/spec/p2p/node.html#sentry-node

[106] Y. Amoussou-Guenou, A. Del Pozzo, M. Potop-Butucaru, and S. Tucci-Piergiovanni, "Correctness and fairness of TenderMint-core blockchains," 2018. [Online]. Available: arXiv:1805.08429.

[107] V. Zamfir. (2015). *Introducing Casper the Friendly Ghost*. [Online]. Available: https://blog.ethereum.org/2015/08/01/introducing-casperfriendly-ghost.

[108] *Casper Proof of Stake Compendium*, Ethereum Found., Zug, Switzerland, 2019. [Online]. Available: https://github.com/ethereum/wiki/wiki/Casper-Proof-of-Stake-compendium

[109] *Sharding Introduction R&D Compendium*, Ethereum Found., Zug, Switzerland, 2019. [Online]. Available: https://github.com/ethereum/wiki/wiki/Sharding-introduction-R&D-compendium

[110] Bitshares 2.0 Documentation. (2015). *The BitShares Organization*. [Online]. Available: http://docs.bitshares.org/bitshares/index.html

[111] *Liskś Consensus Algorithm*. Accessed: Apr. 7, 2019. [Online]. Available: https://lisk.io/documentation/lisk-protocol/consensus

[112] D. Larimer. (2018). *DPOS BFT—Pipelined Byzantine Fault Tolerance*. [Online]. Available: https://medium.com/eosio/dpos-bft-pipelined-byzantine-fault-tolerance-8a0634a270ba

[113] J. Martinez. (2018). *Understanding Proof of Stake: The Nothing at Stake Theory*. [Online]. Available: https://medium.com/coinmonks/understanding-proof-of-stake-the-nothing-at-stake-theory-1f0d71bc027

[114] M. Franklin, "A survey of key evolving cryptosystems," *Int. J. Security Netw.*, vol. 1, nos. 1–2, pp. 46–53, 2006.

[115] V. Buterin. (2014). *Ethereum Foundation*. [Online]. Available: https://blog.ethereum.org/2014/05/15/long-range-attacks-the-serious-problem-with-adaptive-proof-of-work/

[116] P. Gaži, A. Kiayias, and A. Russell, "Stake-bleeding attacks on proof-of-stake blockchains," in *Proc. IEEE Crypto Valley Conf. Blockchain Technol. (CVCBT)*, 2018, pp. 85–92.

[117] *Grinding Attack on Proof of Stake*, Ethereum Found., Zug, Switzerland, 2019. Accessed: Apr. 7, 2019. [Online]. Available: https://github.com/ethereum/wiki/wiki/Proof-of-Stake-FAQs#how-does-validator-selection-work-and-what-is-stake-grinding

[118] *Aura—Authority Round—WiKi, Parity Tech Documentation*. Accessed: Apr. 7, 2019. [Online]. Available: https://wiki.parity.io/Aura

[119] Hyperledger Sawtooth Project. (2018). *The Linux Foundation*. [Online]. Available: https://www.hyperledger.org/projects/sawtooth

[120] V. Costan and S. Devadas, "Intel SGX explained," IACR Cryptol. ePrint Archive, Rep. 2016/086, 2016.

[121] *Introducing ARM Trustzone*, ARM Ltd., Cambridge, U.K., 2018. [Online]. Available: https://developer.arm.com/technologies/trustzone

[122] Hyperledger Sawtooth RFC. (2018). *The Linux Foundation*. [Online]. Available: https://github.com/hyperledger/sawtooth-rfcs/blob/master/text/0019-pbft-consensus.md

[123] F. Brasser *et al.*, "Software grand exposure: SGX cache attacks are practical," in *Proc. 11th USENIX Workshop Offensive Technol. (WOOT)*, 2017, pp. 4–7.

[124] J. Van Bulck *et al.*, "Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution," in *Proc. 27th USENIX Security Symp. (USENIX Security)*, 2018, pp. 991–1008.

[125] O. Weisse *et al.*, "Foreshadow-NG: Breaking the virtual memory abstraction with transient out-of-order execution," USENIX Assoc., Berkeley, CA, USA, Rep. LIRIAS2089352, 2018.

[126] W. Li, S. Andreina, J.-M. Bohli, and G. Karame, "Securing proof-of-stake blockchain protocols," in *Proc. Data Privacy Manag. Cryptocurrencies Blockchain Technol.*, 2017, pp. 297–315.

[127] S. Andreina, J.-M. Bohli, G. O. Karame, W. Li, and G. A. Marson, "PoTS-a secure proof of TEE-stake for permissionless blockchains," IACR Cryptol. ePrint Archive, Rep. 2018/1135, 2018.

[128] A. Juels and B. S. Kaliski, Jr., "PORS: Proofs of retrievability for large files," in *Proc. 14th ACM Conf. Comput. Commun. Security*, 2007, pp. 584–597.

[129] A. Miller, A. Juels, E. Shi, B. Parno, and J. Katz, "Permacoin: Repurposing bitcoin work for data preservation," in *Proc. IEEE Symp. Security Privacy (SP)*, 2014, pp. 475–490.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.

[130] D. Schwartz *et al.*, "The ripple protocol consensus algorithm," vol. 5, San Francisco, CA, USA, Ripple Labs Inc., White Paper, 2014.
[131] D. Mazieres. (2015). *The Stellar Consensus Protocol: A Federated Model for Internet-Level Consensus*. [Online]. Available: http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.696.93&amp=&rep=rep1&amp=&type=pdf
[132] C. Cachin and B. Tackmann, "Asymmetric distributed trust," 2019. [Online]. Available: arXiv:1906.09314.
[133] Y. Sompolinsky, Y. Lewenberg, and A. Zohar, "SPECTRE: A fast and scalable cryptocurrency protocol," IACR Cryptol. ePrint Archive, Rep. 2016/1159, 2016.
[134] Y. Sompolinsky and A. Zohar, "PHANTOM: A scalable blockDAG protocol," IACR Cryptol. ePrint Archive, Rep. 2018/104, 2018.
[135] S. Popov. (2016). *The Tangle*. [Online]. Available: http://www.descryptions.com/Iota.pdf
[136] Tangle Tip Selection. *IOTA Foundation*. Accessed: Apr. 7, 2019. [Online]. Available: https://docs.iota.org/docs/the-tangle/0.1/concepts/tip-selection#in-depth-explanation-of-the-tip-selection-algorithm
[137] A. Churyumov. (2016). *ByteBall: A Decentralized System for Storage and Transfer of Value*. [Online]. Available: https://byteball.org/Byteball.pdf
[138] C. LeMahieu. (2018). *Nano: A Feeless Distributed Cryptocurrency Network*. [Online]. Available: https://nano.org/en/whitepaper
[139] Team Rocket. (2018). *Snowflake to Avalanche: A Novel Metastable Consensus Protocol Family for Cryptocurrencies*. Accessed: Oct. 14, 2019. [Online]. Available: https://pdfs.semanticscholar.org/85ec/19594046bbcfe12137c7c2e3744677129820.pdf
[140] C. Natoli, J. Yu, V. Gramoli, and P. Esteves-Verissimo, "Deconstructing blockchains: A comprehensive survey on consensus, membership and structure," 2019. [Online]. Available: arXiv:1908.08316.
[141] E. B. Sasson *et al.*, "ZeroCash: Decentralized anonymous payments from bitcoin," in *Proc. IEEE Symp. Security Privacy*, 2014, pp. 459–474.
[142] S. Noether, "Ring signature confidential transactions for Monero," IACR Cryptol. ePrint Archive, Rep. 2015/1098, 2015.
[143] R. Canetti, O. Goldreich, and S. Halevi, "The random oracle methodology, revisited," *J. ACM*, vol. 51, no. 4, pp. 557–594, 2004.
[144] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. 42nd IEEE Symp. Found. Comput. Sci.*, 2001, pp. 136–145.



Yang Xiao (Student Member, IEEE) received the B.S. degree from the School of Electrical and Information Engineering, Shanghai Jiao Tong University and the M.S. degree from the Electrical Engineering and Computer Science Department, University of Michigan, Ann Arbor. He is currently pursuing the Ph.D. degree with the Bradley Department of Electrical and Computer Engineering, Virginia Tech, supervised by Prof. W. Lou. His research interests lie in network security, blockchain, and IoT security.



Ning Zhang (Member, IEEE) received the Ph.D. degree from Virginia Tech in 2016. He was with an industry as a Cyber Engineer and a Technical Lead for over ten years. He is currently an Assistant Professor with the Department of Computer Science and Engineering, Washington University in St. Louis. His research focus is system security, which lies at the intersection of security, embedded systems, and computer architecture and software.



Wenjing Lou (Fellow, IEEE) received the Ph.D. degree in electrical and computer engineering from the University of Florida in 2003. From 2003 to 2011, she was a Faculty Member with the Worcester Polytechnic Institute. She has been a Professor with Virginia Tech since 2011. From 2014 to 2017, she was the Program Director of the U.S. National Science Foundation, where she was involved in the Networking Technology and Systems Program and the Secure and Trustworthy Cyberspace Program. Her current research interests focus on privacy protection techniques in networked information systems and cross-layer security enhancement in wireless networks by exploiting intrinsic wireless networking and communication properties.



Y. Thomas Hou (Fellow, IEEE) received the Ph.D. degree from the NYU Tandon School of Engineering (formerly, Polytechnic University) in 1998. In 2002, he joined Virginia Tech, Blacksburg, VA, USA, where he is currently a Bradley Distinguished Professor of electrical and computer engineering. From 1997 to 2002, he was a Member of Research Staff with Fujitsu Laboratories of America, Sunnyvale, CA, USA. He has over 250 papers published in the IEEE/ACM journals and conferences. He holds five U.S. patents. He has authored/coauthored two graduate textbooks *Applied Optimization Methods for Wireless Networks* (Cambridge University Press, in 2014) and *Cognitive Radio Communications and Networks: Principles and Practices* (Academic Press/Elsevier, in 2009). His current research focuses on developing innovative solutions to complex science and engineering problems arising from wireless and mobile networks, and wireless security. His papers were recognized by five Best Paper Awards from the IEEE and two Paper Awards from the ACM. He was named an IEEE Fellow for contributions to modeling and optimization of wireless networks. He was/is on the editorial boards of a number of IEEE and *ACM Transactions* and journals. He is the Steering Committee Chair of the IEEE INFOCOM Conference and a member of the IEEE Communications Society Board of Governors. He is also a Distinguished Lecturer of the IEEE Communications Society.

Authorized licensed use limited to: SOUTHWEST JIAOTONG UNIVERSITY. Downloaded on March 01,2026 at 07:40:48 UTC from IEEE Xplore. Restrictions apply.